



UNIVERSITAS INDONESIA

JUDUL SKRIPSI/THESIS/DISERTASI

SKRIPSI

MOHAMMAD RAFFY ZEIDAN

2206051462

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

PROGRAM STUDI STATISTIKA

DEPOK

2026



UNIVERSITAS INDONESIA

JUDUL SKRIPSI/THESIS/DISERTASI

SKRIPSI

**Diajukan sebagai salah satu syarat untuk memperoleh gelar
Sarjana Statistika**

MOHAMMAD RAFFY ZEIDAN

2206051462

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

PROGRAM STUDI STATISTIKA

DEPOK

MEI 2026

HALAMAN PERSETUJUAN

Judul : Judul Skripsi/Thesis/Disertasi
Nama : Mohammad Raffy Zeidan
NPM : 2206051462

Laporan Skripsi ini telah diperiksa dan disetujui.

XX Juli 2019

Prof. ???
Pembimbing Skripsi

HALAMAN PERNYATAAN ORISINALITAS

**Skripsi ini adalah hasil karya saya sendiri,
dan semua sumber baik yang dikutip maupun dirujuk
telah saya nyatakan dengan benar.**

Nama : Mohammad Raffy Zeidan
NPM : 2206051462
Tanda Tangan :

Tanggal : XX Juli 2019

HALAMAN PENGESAHAN

Skripsi ini diajukan oleh :
Nama : Mohammad Raffy Zeidan
NPM : 2206051462
Program Studi : Statistika
Judul Skripsi : Judul Skripsi/Thesis/Disertasi

Telah berhasil dipertahankan di hadapan Dewan Penguji dan diterima sebagai bagian persyaratan yang diperlukan untuk memperoleh gelar Sarjana Statistika pada Program Studi Statistika, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Indonesia.

DEWAN PENGUJI

Pembimbing : Prof. ??? ()

Penguji : Prof. XXX ()

Penguji : Prof. XXXX ()

Penguji : Prof. XXXXXX ()

@todo

Jangan lupa mengisi nama para penguji.

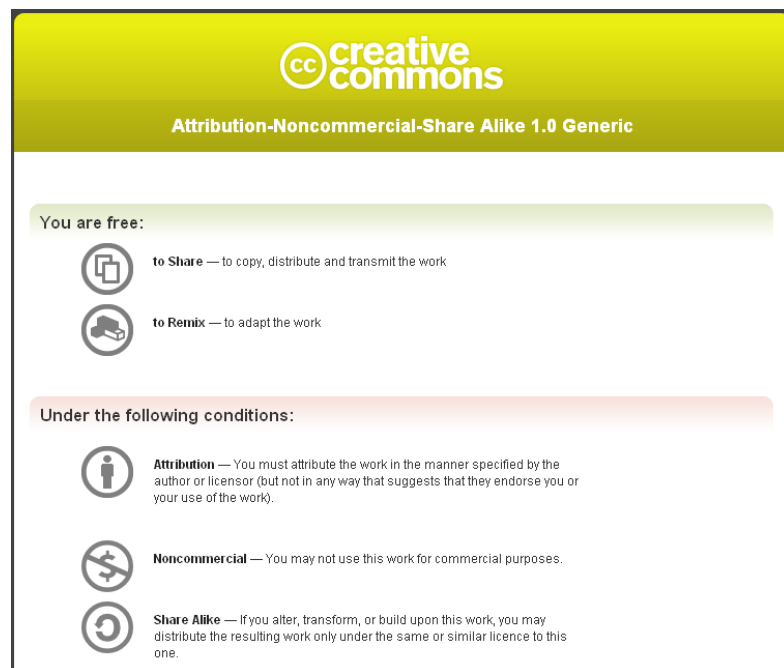
Ditetapkan di : Depok
Tanggal : XX Juli 2019

KATA PENGANTAR

Template ini disediakan untuk orang-orang yang berencana menggunakan $\text{\LaTeX}$ untuk membuat dokumen tugas akhirnya. Mengapa $\text{\LaTeX}$? Ada banyak hal mengapa menggunakan $\text{\LaTeX}$, diantaranya:

1. $\text{\LaTeX}$ membuat kita jadi lebih fokus terhadap isi dokumen, bukan tampilan atau halaman.
2. $\text{\LaTeX}$ memudahkan dalam penulisan persamaan matematis.
3. Adanya otomatis dalam penomoran caption, bab, subbab, subsubbab, referensi, dan rumus.
4. Adanya otomatisasi dalam pembuatan daftar isi, daftar gambar, dan daftar tabel.
5. Adanya kemudahan dalam memberikan referensi dalam tulisan dengan menggunakan label. Cara ini dapat meminimalkan kesalahan pemberian referensi.

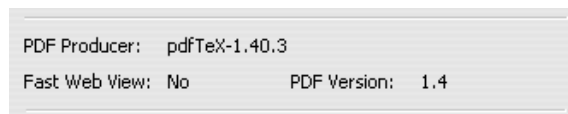
Template ini bebas digunakan dan didistribusikan sesuai dengan aturan *Creative Common License 1.0 Generic*, yang secara sederhana berisi:



Gambar 1: *Creative Common License 1.0 Generic*

Gambar 1 diambil dari http://creativecommons.org/licenses/by-nc-sa/1.0/deed.en_CA. Jika ingin mengetahui lebih lengkap mengenai *Creative Common License 1.0 Generic*, silahkan buka <http://creativecommons.org/licenses/by-nc-sa/1.0/legalcode>. Seluruh dokumen yang dibuat dengan menggunakan template ini sepenuhnya menjadi hak milik pembuat dokumen dan bebas didistribusikan sesuai dengan keperluan masing-masing. Lisensi hanya berlaku jika ada orang yang membuat template baru dengan menggunakan template ini sebagai dasarnya.

Dokumen ini dibuat dengan $\text{\LaTeX}$ juga. Untuk meyakinkan Anda, coba lihat properti dari dokumen ini dan Anda akan menemukan bagian seperti Gambar 2. Dokumen ini dimaksudkan untuk memberikan gambaran kepada Anda seperti apa mudahnya menggunakan $\text{\LaTeX}$ dan juga memperlihatkan betapa bagus dokumen yang dihasilkan. Seluruh url yang Anda temukan dapat Anda klik. Seluruh referensi yang ada juga dapat diklik. Untuk mengerti template yang disediakan, Anda tetap harus membuka kode $\text{\LaTeX}$ dan bermain-main dengannya. Penjelasan dalam PDF ini masih bersifat gambaran dan tidak begitu mendetail, dapat dianggap sebagai pengantar singkat. Jika Anda merasa kesulitan dengan template ini, mungkin ada baiknya Anda belajar sedikit dasar-dasar $\text{\LaTeX}$.



Gambar 2: Dokumen Dibuat dengan PDF $\text{\LaTeX}$

Semoga template ini dapat membantu orang-orang yang ingin mencoba menggunakan $\text{\LaTeX}$. Semoga template ini juga tidak berhenti disini dengan ada kontribusi dari para penggunanya. Kami juga ingin berterima kasih kepada Andreas Febrian, Lia Sadita, Fahrurrozi Rahman, Andre Tampubolon, dan Erik Dominikus atas kontribusinya dalam template ini.

Depok, 27 Februari 2019

Mohammad Raffy Zeidan

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai sivitas akademik Universitas Indonesia, saya yang bertanda tangan di bawah ini:

Nama : Mohammad Raffy Zeidan
NPM : 2206051462
Program Studi : Statistika
Fakultas : Matematika dan Ilmu Pengetahuan
Alam
Jenis Karya : Skripsi

demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Universitas Indonesia **Hak Bebas Royalti Noneksklusif** (*Non-exclusive Royalty Free Right*) atas karya ilmiah saya yang berjudul:

Judul Skripsi/Thesis/Disertasi

beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini Universitas Indonesia berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (*database*), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Depok
Pada tanggal : XX Juli 2019
Yang menyatakan

(Mohammad Raffy Zeidan)

ABSTRAK

Nama : Mohammad Raffy Zeidan
Program Studi : Statistika
Judul : Judul Skripsi/Thesis/Disertasi
Pembimbing : Prof. ???

Tesis ini membahas kemampuan mahasiswa Fakultas Psikologi UI dalam mencari dan menggunakan informasi secara efektif dalam konteks *active learning* dan *self regulated learning* selama mereka mengikuti Program Pendidikan Dasar Pendidikan Tinggi. Penelitian ini adalah penelitian kualitatif dengan desain deskriptif. Hasil penelitian menyarankan bahwa perpustakaan perlu dilibatkan dalam pengembangan kurikulum; materi pendidikan pemakai perpustakaan harus dikembangkan sesuai dengan komponen-komponen yang ada dalam *information literacy*; perpustakaan juga harus menyediakan sarana dan fasilitas yang mendukung peningkatan *literacy* mahasiswa.

Kata kunci:

Informasi, *information literacy*, *information skills*

ABSTRACT

Name : Mohammad Raffy Zeidan
Study Program : Statistika
Title : Unknown Title for Final Report/Thesis/Disertation
Counsellor : Prof. ???

The focus of this study is the freshman student of Faculty of Psychology at University of Indonesia experience of acquiring, evaluating and using information, when they enroll in “Program Dasar Pendidikan Tinggi (PDPT)”. The purpose of this study is to understand how freshman students acquire, evaluate and use information. Knowing this will allow library to identify changes should be made to improve user education program at University of Indonesia. This research is qualitative descriptive interpretive. The data were collected by means of deep interview. The researcher suggests that library should improve the user education program and provide facilities which can help students to be information literate.

Key words:

Information literacy, information skills, information

DAFTAR ISI

HALAMAN JUDUL	i
LEMBAR PERSETUJUAN	ii
LEMBAR PERNYATAAN ORISINALITAS	iii
LEMBAR PENGESAHAN	iv
KATA PENGANTAR	v
LEMBAR PERSETUJUAN PUBLIKASI ILMIAH	vi
ABSTRAK	viii
DAFTAR ISI	x
DAFTAR GAMBAR	xiii
DAFTAR TABEL	xiv
1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Permasalahan	5
1.3 Tujuan Penelitian	6
1.4 Manfaat Penelitian	6
1.5 Ruang Lingkup Penelitian	7
2 LANDASAN TEORI	8
2.1 Analisis Sentimen	8
2.2 Pendeteksian Topik	9
2.3 Neural Network	9
2.3.1 Fungsi Aktivasi	11
2.3.1.1 Sigmoid	12
2.3.1.2 Rectified Linear Unit (ReLU)	12
2.3.1.3 Gaussian Error Linear Unit (GELU)	12
2.3.2 Fungsi <i>Loss</i>	13

2.3.3	Backpropagation	13
2.3.4	Teknik Optimasi	16
2.3.4.1	Stochastic Gradient Descent (SGD)	16
2.3.4.2	<i>Adaptive Moment Estimation</i> (Adam)	16
2.3.5	Regularisasi	17
2.3.6	Metrik Evaluasi	18
2.4	Autoencoder	19
2.4.1	Encoder dan Decoder	20
2.4.2	Fungsi Objektif	20
2.5	Bidirectional Encoder Representations from Transformers (BERT) .	21
2.5.1	Arsitektur BERT	21
2.5.2	Mekanisme <i>Self-Attention</i>	22
2.5.3	Representasi Input BERT	23
2.5.4	Ekstraksi Fitur dengan BERT	24
2.6	Multi-Task Learning	24
2.6.1	Formulasi MTL	25
2.6.2	Hard Parameter Sharing	26
2.6.3	Soft Parameter Sharing	27
2.7	Clustering	28
2.7.1	K-Means Clustering	29
2.8	Interpretabilitas Model <i>Deep Learning</i>	30
2.8.1	<i>Occlusion-based Token Attribution</i>	31
2.8.2	<i>Integrated Gradients</i>	32
2.9	Interpretabilitas Model <i>Deep Learning</i>	33

3 METODE MODEL JOINT SENTIMENT-TOPIC BERBASIS MULTITASK LEARNING

34

3.1	<i>Bidirectional Encoder Representations from Transformers</i> (BERT) Sebagai Model Representasi Teks	34
3.2	Model <i>Joint Sentiment-Topic</i> Berbasis <i>Multitask Learning</i>	36
3.3	<i>Autoencoder</i>	37
3.4	Analisis Sentimen Menggunakan <i>Sentiment Head</i>	38
3.5	<i>Deep Embedded Clustering</i> (DEC)	39
3.6	Interpretasi Model dengan Analisis Atribusi Token	40
3.6.1	Implementasi <i>Occlusion-based Token Attribution</i>	40
3.6.2	Agregasi Atribusi Token per Kluster Topik	41
3.6.3	Implementasi <i>Integrated Gradients</i>	42
3.7	Evaluasi Topik	42

3.7.1	<i>Topic Coherence</i> Berbasis NPMI	43
3.7.2	<i>Topic Diversity</i>	44
3.7.3	<i>Topic Coverage</i> dan Keseimbangan Klaster	44
4	HASIL DAN SIMULASI DAN PEMBAHASAN	46
5	KESIMPULAN	47
5.1	Kesimpulan	47
5.2	Saran	47

DAFTAR GAMBAR

Gambar 1	<i>Creative Common License 1.0 Generic</i>	v
Gambar 2	Dokumen Dibuat dengan PDFLatex	vi
Gambar 2.1	Ilustrasi <i>neural network</i> secara biologis (Aggarwal 2018) .	10
Gambar 2.2	<i>Artificial Neural Network</i> dengan representasi bobot sinapsis (Aggarwal 2018)	10
Gambar 2.3	Arsitektur <i>feed-forward neural network</i> (Aggarwal 2018) .	11
Gambar 2.4	Grafik fungsi aktivasi: Sigmoid, ReLU, dan GELU	13
Gambar 2.5	Struktur <i>multi-layer perceptron</i> pada jalur <i>input-output</i> (Aggarwal 2018)	14
Gambar 2.6	Ilustrasi fenomena <i>underfitting</i> dan <i>overfitting</i>	18
Gambar 2.7	Arsitektur Autoencoder menunjukkan proses <i>encoding</i> dan <i>decoding</i> (Aggarwal 2018)	20
Gambar 2.8	Arsitektur BERT dengan beberapa lapisan <i>Transformer</i> <i>Encoder</i> Devlin <i>et al.</i> 2018	22
Gambar 2.9	Representasi <i>input</i> BERT yang terdiri dari <i>token</i> , <i>segment</i> , dan <i>position embedding</i> Devlin <i>et al.</i> 2018	24
Gambar 2.10	Ilustrasi arsitektur <i>hard parameter sharing</i> dalam <i>Multi- Task Learning</i> (Ruder 2017)	27
Gambar 2.11	Ilustrasi arsitektur <i>soft parameter sharing</i> . Setiap tugas memiliki model tersendiri, namun parameter antar model diregularisasi agar tetap serupa (Ruder 2017)	28
Gambar 3.1	Alur metode analisis sentimen dan pendeteksian topik . . .	34
Gambar 3.2	Representasi teks menggunakan BERT ((Devlin <i>et al.</i> 2018); telah diolah kembali).	35

DAFTAR TABEL

Tabel 2.1	Algoritma K-Means <i>Clustering</i>	30
Tabel 3.1	Algoritma <i>Deep Embedded Clustering</i> (DEC)	40

DAFTAR KODE

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dunia kontemporer dibanjiri oleh arus deras informasi tekstual yang dihasilkan setiap saat, bersumber dari platform media sosial, ulasan daring, artikel berita, repositori ilmiah, dan banyak lagi. Kumpulan data tekstual masif ini, yang sebagian besar tidak terstruktur, mengandung potensi pengetahuan yang signifikan mengenai sentimen masyarakat, dinamika pasar, hingga tren diskursus publik. Akan tetapi, volume dan keragaman data tersebut menjadikan analisis manual suatu hal yang tidak praktis dan efisien. Menjawab tantangan ini, disiplin *Natural Language Processing* (NLP), sebagai bagian dari kecerdasan artifisial yang menjembatani komunikasi antara mesin dan bahasa alami manusia, hadir menawarkan metodologi komputasional untuk mengolah dan memahami data linguistik dalam skala besar.

Dalam spektrum kapabilitas NLP, dua teknik yang memiliki relevansi tinggi untuk mengekstrak makna dari himpunan data tekstual adalah analisis sentimen (*Sentiment Analysis*) dan pemodelan topik (*Topic Modeling*) (Blei, Ng, and Jordan 2003). Analisis sentimen secara khusus dirancang untuk menyelidiki dan menentukan muatan subjektif atau nada emosional yang terkandung dalam suatu teks—menggolongkannya sebagai ekspresi positif, negatif, ataupun netral. Pengaplikasian teknik ini sangat berharga untuk melacak citra merek, menganalisis tanggapan konsumen, memetakan opini publik terkait isu tertentu, serta mengidentifikasi ujaran problematik. Kemampuan mengukur sentimen secara kolektif memberikan landasan bagi entitas bisnis maupun riset untuk memahami persepsi dan mengambil langkah strategis.

Di sisi lain, pemodelan topik menangani tugas komplementer, yakni mengungkap tema-tema atau pokok bahasan esensial yang tersembunyi (*latent thematic structures*) di dalam koleksi dokumen berjumlah besar, atau sering disebut *corpora* (Blei, Ng, and Jordan 2003). Berbeda dengan fokus pada nuansa afektif per teks, tujuan utama pemodelan topik adalah menyajikan intisari tematik dari keseluruhan data. Kegunaannya terbentang luas, mulai dari fasilitasi eksplorasi dan pengorganisasian arsip digital, identifikasi pola baru dalam korpus literatur atau berita, hingga penyempurnaan sistem rekomendasi konten dan pemahaman struktur narasi dalam

skala makro. Dengan mengabstraksikan topik-topik dominan, metode ini membantu mereduksi kompleksitas informasi dan menyajikan gambaran umum konten. Walaupun memiliki fokus yang berbeda, kedua pendekatan ini—analisis sentimen dan pemodelan topik—berpotensi menghasilkan pemahaman yang lebih komprehensif jika diintegrasikan, menyingkap tidak hanya *apa* subjek pembicaraan, tetapi juga *bagaimana* pandangan atau perasaan terhadap subjek tersebut.

Pendekatan tradisional dalam pemodelan topik, seperti *Latent Dirichlet Allocation* (LDA), umumnya berbasis probabilistik dan mengandalkan metode faktorisasi matriks untuk mengidentifikasi distribusi kata dalam topik (Blei, Ng, and Jordan 2003; Blei 2012). Meskipun metode ini telah terbukti efektif dalam banyak kasus, keterbatasannya dalam menangkap hubungan semantik yang kompleks dalam teks membuatnya kurang optimal untuk tugas yang lebih kompleks, seperti klasifikasi sentimen yang dilakukan secara simultan dengan deteksi topik. Salah satu kelemahan utama pendekatan probabilistik seperti LDA adalah skalabilitasnya yang terbatas, terutama saat diaplikasikan pada dataset berukuran besar dengan dimensi fitur tinggi (Boyd-Graber, Hu, and Mimno 2017). Selain itu, model berbasis probabilistik sering kali memerlukan waktu komputasi yang signifikan dan sulit diadaptasi untuk menangani data teks modern yang bersifat dinamis dan tidak terstruktur (McAuliffe and Blei 2008).

Dalam klasifikasi sentimen, pendekatan berbasis pembelajaran mendalam (*deep learning*) telah menunjukkan keunggulan signifikan dibandingkan metode tradisional. Salah satu model yang paling efektif adalah *Bidirectional Encoder Representations from Transformers* (BERT), yang mampu memahami konteks kata dalam teks dengan mempertimbangkan hubungan antar kata di kedua arah (*bidirectional*) (Devlin *et al.* 2018). BERT telah banyak digunakan dalam tugas-tugas NLP dan terbukti unggul dalam klasifikasi sentimen, menghasilkan akurasi yang lebih tinggi dibandingkan pendekatan berbasis fitur atau metode statistik konvensional (Sun *et al.* 2019).

Seiring dengan berkembangnya model deep learning, pendekatan berbasis jaringan saraf telah terbukti lebih unggul dalam menangani data teks yang tidak terstruktur dibandingkan metode berbasis probabilistik seperti LDA (Mikolov, K. Chen *et al.* 2013). Model neural network dapat mengekstrak representasi semantik yang lebih kaya dan menangkap hubungan kontekstual antar kata dalam teks dengan lebih baik (Yinhan Zhang, H. Zhao, and M. Li 2020). Selain itu, pendekatan neural network dapat mengatasi keterbatasan skalabilitas pada metode probabilistik

karena dapat memanfaatkan arsitektur yang lebih efisien seperti *transformers* dan teknik embedding untuk mengurangi dimensi data (Vaswani *et al.* 2017).

Salah satu pendekatan berbasis neural network yang semakin populer dalam pemodelan topik adalah **BERT-Topic**. Berbeda dengan LDA yang hanya mengandalkan distribusi kata untuk menentukan topik, BERT-Topic memanfaatkan representasi teks dari model transformer seperti BERT untuk menangkap hubungan semantik yang lebih dalam antar kata (Grootendorst 2022). Metode ini menggabungkan embedding berbasis transformer dengan teknik klusterisasi seperti *UMAP* dan *HDBSCAN* untuk mengidentifikasi topik secara lebih akurat dan stabil, bahkan dalam dataset berukuran besar atau dengan struktur yang kompleks. Keunggulan utama BERT-Topic dibandingkan LDA adalah kemampuannya dalam menangani variasi bahasa yang lebih luas, memahami sinonim dan kontekstualisasi kata, serta tidak bergantung pada asumsi distribusi kata seperti pada metode probabilistik (Grootendorst 2022).

Namun demikian, sebagian besar metode pemodelan topik modern seperti BERT-Topic masih berfokus pada pemisahan topik semata tanpa mempertimbangkan aspek lain seperti sentimen atau struktur laten yang lebih dalam secara eksplisit. Selain itu, integrasi antara pemodelan topik dan klasifikasi sentimen masih sering dilakukan secara terpisah, sehingga informasi semantik yang berharga dalam teks belum sepenuhnya dimanfaatkan secara terpadu. Pemisahan ini mengakibatkan redundansi komputasi dan hilangnya peluang untuk memanfaatkan sinergi antara kedua tugas dalam satu arsitektur model yang kohesif.

Dalam konteks pembelajaran representasi untuk data teks berdimensi tinggi, *Autoencoder* telah menunjukkan kemampuannya sebagai teknik pembelajaran tanpa supervisi (*unsupervised learning*) yang efektif untuk reduksi dimensi dan ekstraksi fitur (Hinton and Salakhutdinov 2006). Autoencoder bekerja dengan cara mempelajari representasi laten (*latent representation*) yang lebih kompak dari data input melalui proses encoding dan decoding. Arsitektur ini memaksa model untuk mengekstrak informasi paling esensial dari data dengan mengompresi input ke dalam ruang dimensi yang lebih rendah, kemudian merekonstruksi kembali data asli dari representasi tersebut. Proses ini memungkinkan model untuk menangkap struktur intrinsik dan pola-pola penting dalam data teks, sekaligus mengurangi noise dan redundansi fitur.

Berbeda dengan teknik reduksi dimensi tradisional seperti *Principal Component Analysis* (PCA) yang bersifat linear, Autoencoder mampu menangkap hubungan

non-linear yang kompleks dalam data (Hinton and Salakhutdinov 2006). Hal ini sangat penting dalam pemrosesan teks, di mana hubungan antar kata dan dokumen seringkali bersifat non-linear dan kontekstual. Autoencoder juga memiliki fleksibilitas arsitektur yang memungkinkan penyesuaian kedalaman jaringan sesuai dengan kompleksitas data dan tugas yang dihadapi. Dalam konteks NLP, penggunaan Autoencoder telah terbukti efektif untuk berbagai aplikasi, termasuk pembelajaran representasi dokumen, deteksi anomali dalam teks, dan ekstraksi fitur untuk tugas-tugas downstream seperti klasifikasi dan klusterisasi (Vincent *et al.* 2010).

Untuk menjawab keterbatasan integrasi antara pemodelan topik dan klasifikasi sentimen, penelitian ini bertujuan untuk mengembangkan pendekatan pemodelan topik yang lebih terintegrasi melalui kombinasi metode representasi laten berbasis *Autoencoder* dan teknik *Deep Embedded Clustering* (DEC). Dengan memanfaatkan Autoencoder sebagai komponen encoder untuk menghasilkan representasi teks yang lebih efisien dan bermakna, serta DEC untuk melakukan klusterisasi yang mempertahankan struktur semantik dalam ruang laten, model diharapkan mampu mengidentifikasi topik-topik secara lebih akurat dan representatif. Autoencoder dalam penelitian ini berfungsi sebagai fondasi untuk pembelajaran representasi yang robust, yang kemudian dioptimalkan lebih lanjut melalui proses klusterisasi DEC untuk menghasilkan pembagian topik yang lebih koheren dan terpisah dengan baik.

Lebih lanjut, penelitian ini mengusulkan pengembangan arsitektur *multi-task learning* (MTL) yang memungkinkan integrasi klasifikasi sentimen dan pemodelan topik secara bersamaan dalam satu kerangka kerja. Pendekatan MTL memiliki keunggulan dalam memanfaatkan hubungan implisit antara tugas-tugas terkait untuk meningkatkan generalisasi model (Caruana 1997). Dalam konteks penelitian ini, klasifikasi sentimen dan pemodelan topik memiliki keterkaitan semantik yang kuat—sentimen suatu teks seringkali berkorelasi dengan topik yang dibahas, dan pemahaman topik dapat membantu dalam identifikasi sentimen yang lebih akurat. Dengan berbagi representasi laten yang sama melalui Autoencoder, kedua tugas dapat saling memperkaya dan menghasilkan performa yang lebih baik dibandingkan jika dilatih secara terpisah.

Pendekatan ini diharapkan dapat meningkatkan efisiensi dan efektivitas analisis teks dengan memanfaatkan representasi laten yang sama untuk dua tugas sekaligus, mengurangi redundansi komputasi, serta memberikan kontribusi terhadap pengembangan model-model NLP yang lebih adaptif dan skalabel. Dengan mengintegrasikan Autoencoder untuk pembelajaran representasi, DEC untuk klusterisasi

topik, dan arsitektur MTL untuk klasifikasi sentimen simultan, penelitian ini berupaya menghadirkan solusi komprehensif yang mengatasi keterbatasan pendekatan-pendekatan sebelumnya dalam pemrosesan bahasa alami.

1.2 Rumusan Permasalahan

Dalam pemrosesan bahasa alami (*Natural Language Processing*, NLP), klasifikasi sentimen dan pemodelan topik merupakan dua tugas utama yang sering dilakukan secara terpisah. Pendekatan tradisional seperti *Latent Dirichlet Allocation* (LDA) berbasis probabilistik memiliki keterbatasan dalam menangkap hubungan semantik yang kompleks, sehingga kurang optimal untuk tugas yang membutuhkan pemahaman konteks yang lebih dalam.

Selain itu, pemisahan antara tugas klasifikasi sentimen dan pemodelan topik menyebabkan hilangnya potensi peningkatan performa yang bisa diperoleh dengan memanfaatkan hubungan alami antar tugas tersebut. Model berbasis deep learning, seperti *Bidirectional Encoder Representations from Transformers* (BERT), telah terbukti unggul dalam tugas klasifikasi sentimen, namun pendekatan ini belum sepenuhnya diintegrasikan dengan pemodelan topik dalam satu framework yang efisien.

Beberapa permasalahan utama yang ingin dijawab dalam penelitian ini adalah:

1. Bagaimana mengembangkan model berbasis *Fully Neural Network* yang dapat menangani klasifikasi sentimen dan pemodelan topik secara simultan?
2. Bagaimana pengaruh pendekatan *Multi-Task Learning* (MTL) terhadap performa model dibandingkan dengan model single-task?
3. Bagaimana penerapan *Autoencoder* dapat membantu mengatasi permasalahan dimensi tinggi dalam data teks?
4. Sejauh mana model yang dikembangkan dapat meningkatkan efisiensi dan akurasi dibandingkan dengan metode sebelumnya, seperti analisis sentimen menggunakan EFCM dan model sentimen BERT?

Melalui penelitian ini, diharapkan dapat diperoleh solusi yang lebih efisien dan akurat dalam menangani kedua tugas secara simultan dengan memanfaatkan pendekatan pembelajaran mendalam berbasis *Fully Neural Network*.

1.3 Tujuan Penelitian

Penelitian ini bertujuan untuk mengembangkan dan mengevaluasi model berbasis *Fully Neural Network* yang mampu melakukan klasifikasi sentimen dan pemodelan topik secara simultan menggunakan pendekatan *Multi-Task Learning* (MTL). Dengan pendekatan ini, model diharapkan dapat memanfaatkan hubungan antara kedua tugas tersebut untuk meningkatkan performa keseluruhan dan efisiensi dalam pemrosesan bahasa alami (*Natural Language Processing*, NLP).

Secara spesifik, penelitian ini memiliki beberapa tujuan utama sebagai berikut:

1. **Mengembangkan arsitektur berbasis *Fully Neural Network*** yang dapat secara bersamaan melakukan klasifikasi sentimen dan pemodelan topik dalam satu model terpadu. Dengan integrasi ini, diharapkan model mampu menangkap hubungan semantik yang lebih dalam dan menghasilkan representasi teks yang lebih bermakna.
2. **Menganalisis pengaruh *Multi-Task Learning* (MTL) terhadap efisiensi dan performa model** dalam menangani kedua tugas secara simultan. Evaluasi akan dilakukan untuk memahami bagaimana strategi berbagi parameter antara dua tugas ini dapat meningkatkan akurasi, stabilitas, dan efisiensi model dibandingkan dengan pendekatan **single-task**.
3. **Menerapkan *Autoencoder* untuk reduksi dimensi**, guna meningkatkan representasi fitur dan mengoptimalkan performa model *Fully Neural Network*. Autoencoder akan digunakan untuk mengekstraksi informasi latar belakang dari teks dengan lebih efektif, mengurangi redundansi fitur, dan membantu model dalam menangkap struktur data yang lebih informatif.

Melalui penelitian ini, diharapkan dapat diperoleh model yang tidak hanya lebih efisien dan akurat, tetapi juga lebih tangguh dalam menangani klasifikasi sentimen dan pemodelan topik secara simultan. Implementasi *Fully Neural Network* dengan *Multi Task Learning* (MTL) dan Autoencoder diharapkan dapat memberikan kontribusi dalam peningkatan teknik pemrosesan bahasa alami, terutama dalam konteks analisis teks skala besar dan kompleks.

1.4 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat baik secara teoretis maupun praktis, yang diuraikan sebagai berikut:

1. Manfaat Teoretis

Penelitian ini diharapkan mampu memperluas wawasan dalam bidang *Natural Language Processing* (NLP), terutama dalam hal perancangan model yang menggabungkan analisis sentimen dan pemodelan topik dalam satu arsitektur secara simultan. Penggunaan *Deep Embedded Clustering* (DEC) sebagai komponen pemodelan topik berbasis *unsupervised learning* dalam kerangka *joint model* diharapkan dapat menambah khasanah penelitian mengenai integrasi dua tugas yang berbeda dalam satu sistem pembelajaran. Penelitian ini diharapkan dapat menjadi acuan bagi penelitian berikutnya dalam membangun sistem analisis teks yang lebih terintegrasi dan efisien.

2. Manfaat Praktis

Hasil penelitian ini diharapkan dapat digunakan sebagai alat bantu dalam memahami isi ulasan teks secara lebih menyeluruh, baik dari sisi sentimen maupun topik yang terkandung di dalamnya, dalam satu kali proses inferensi. Sistem yang dikembangkan berpotensi diterapkan pada berbagai domain, seperti pemantauan ulasan produk, evaluasi layanan pelanggan, hingga analisis tren opini publik di platform digital. Pendekatan *joint model* ini juga diharapkan dapat mengurangi beban komputasi dibandingkan menjalankan kedua analisis secara terpisah.

1.5 Ruang Lingkup Penelitian

Batasan dalam penelitian ini ditetapkan sebagai berikut:

1. Penelitian ini menggunakan analisis sentimen biner yang terdiri atas dua kelas, yaitu positif dan negatif.
2. Data yang digunakan merupakan *dataset* IMDB dan Yelp yang berupa kumpulan teks ulasan, yang diperoleh melalui platform Hugging Face, masing-masing pada dataset IMDB dan dataset Yelp Polarity.
3. Jumlah data ulasan yang digunakan dalam penelitian ini dibatasi sebanyak 30.000 ulasan untuk masing-masing *dataset*, baik IMDB maupun Yelp.

BAB 2

LANDASAN TEORI

Bab ini memaparkan landasan teori yang menjadi fondasi penelitian. Pembahasan dimulai dari konsep *analisis sentimen* beserta tingkatan dan pendekatannya, dilanjutkan dengan *pendeteksian topik* dan berbagai metodenya. Selanjutnya diuraikan dasar-dasar *neural network*, mulai dari arsitektur, fungsi aktivasi, *backpropagation*, hingga metrik evaluasi. Bab ini juga membahas *Bidirectional Encoder Representations from Transformers* (BERT) sebagai model representasi teks berbasis *transformer*, *autoencoder* sebagai komponen kompresi representasi, serta *multi-task learning* sebagai kerangka pelatihan model secara simultan untuk beberapa tugas sekaligus.

2.1 Analisis Sentimen

Di era digital, setiap pengguna internet dapat dengan bebas mengekspresikan pendapat dan ulasan mereka terhadap berbagai hal melalui platform daring seperti media sosial, forum diskusi, hingga situs ulasan produk. Fenomena ini menghasilkan tumpukan data opini yang sangat besar dan terus bertambah, sehingga muncul kebutuhan akan metode komputasional yang mampu mengolahnya secara otomatis.

Analisis sentimen, yang juga dikenal sebagai *opinion mining*, merupakan bidang kajian dalam pemrosesan bahasa alami (*Natural Language Processing/NLP*) yang berfokus pada identifikasi dan ekstraksi opini, sentimen, serta emosi yang terkandung dalam teks (B. Liu 2012). Secara umum, analisis sentimen berupaya mengklasifikasikan kecenderungan opini ke dalam kategori polaritas, yakni positif, negatif, atau netral. Analisis ini dapat dilakukan pada berbagai tingkatan unit teks, mulai dari keseluruhan dokumen, kalimat per kalimat, hingga aspek-aspek tertentu dalam sebuah teks. Pada penelitian ini, analisis dilakukan pada tingkat dokumen, artinya setiap ulasan secara keseluruhan diklasifikasikan ke dalam satu label sentimen. Pendekatan ini sesuai karena dataset yang digunakan, yakni Yelp dan IMDB, masing-masing berisi ulasan yang ditulis oleh satu pengguna dan membahas satu entitas tunggal.

Secara metodologis, analisis sentimen dapat dilakukan melalui dua pendekatan utama. Pendekatan berbasis leksikon memanfaatkan kamus sentimen yang berisi kata-kata beserta nilai polaritasnya untuk menentukan sentimen teks secara lang-

sung. Sementara itu, pendekatan berbasis *machine learning* melatih model menggunakan data berlabel agar model dapat mempelajari pola bahasa yang berkorelasi dengan sentimen tertentu (Pang, L. Lee, and Vaithyanathan 2002). Pendekatan berbasis *machine learning* terbukti lebih unggul dalam menangani variasi bahasa, konteks, dan domain yang beragam, terutama pada data berskala besar. Oleh karena itu, penelitian ini menggunakan pendekatan berbasis *machine learning* sebagai landasan utama.

2.2 Pendeteksian Topik

Topik, dalam pengertian umum, merujuk pada tema utama yang menjadi inti dari sebuah teks atau dokumen. Dalam konteks pemrosesan bahasa alami, pendeteksian topik adalah proses komputasional yang bertujuan mengidentifikasi tema-tema tersembunyi di dalam kumpulan dokumen teks tanpa adanya label yang diberikan sebelumnya (Blei 2012). ^{buang saja bagian ini} Dalam kerangka ini, setiap topik direpresentasikan sebagai distribusi probabilitas atas sekumpulan kata, sehingga topik yang terdeteksi dapat dipahami maknanya melalui kata-kata yang paling sering muncul di dalamnya. Karena tidak memerlukan label, pendeteksian topik secara alami tergolong ke dalam paradigma *unsupervised learning*, yakni pendekatan di mana model belajar dari pola yang ada dalam data itu sendiri tanpa bimbingan dari anotasi manusia.

Salah satu metode ^{DL/ML} modern yang banyak digunakan adalah BERTopic (Groo-tendorst 2022), yang menggabungkan representasi kontekstual dari model bahasa besar dengan teknik *clustering* untuk menghasilkan topik-topik yang kaya makna semantis.

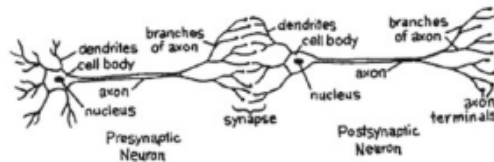
Pada penelitian ini, pendekatan berbasis *clustering* digunakan sebagai metode utama pendeteksian topik. Ide dasarnya adalah mengelompokkan dokumen-dokumen yang memiliki kedekatan makna ke dalam satu klaster, di mana setiap klaster kemudian diinterpretasikan sebagai sebuah topik. Proses pengelompokan ini dilakukan pada ruang representasi laten yang dihasilkan oleh Autoencoder, yakni sebuah model jaringan saraf yang belajar memampatkan informasi teks ke dalam bentuk yang lebih ringkas namun tetap merepresentasikan maknanya.

2.3 Neural Network

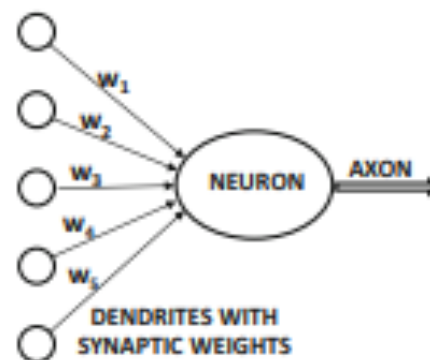
Jaringan saraf tiruan (*Artificial Neural Network/ANN*) merupakan paradigma komputasi yang terinspirasi dari sistem saraf biologis manusia. Sebagaimana neuron biologis saling terhubung melalui sinapsis, ANN terdiri dari unit-unit komputasi

yang saling terkoneksi melalui bobot yang dapat disesuaikan. Proses pembelajaran berlangsung melalui penyesuaian bobot-bobot tersebut berdasarkan data pelatihan yang diberikan.

Secara umum, arsitektur ANN tersusun atas tiga komponen utama (Silva *et al.* 2017). Pertama, *input layer*, yaitu lapisan yang menerima data mentah dari luar sistem dan meneruskannya ke lapisan berikutnya tanpa melakukan transformasi apapun. Kedua, *hidden layer*, yaitu lapisan yang menerapkan fungsi aktivasi untuk mengidentifikasi pola kompleks dari data. Ketiga, *output layer*, yaitu lapisan yang menghasilkan prediksi akhir berdasarkan representasi yang telah dibangun oleh lapisan-lapisan sebelumnya.



Gambar 2.1: Ilustrasi *neural network* secara biologis (Aggarwal 2018)



Gambar 2.2: *Artificial Neural Network* dengan representasi bobot sinapsis (Aggarwal 2018)

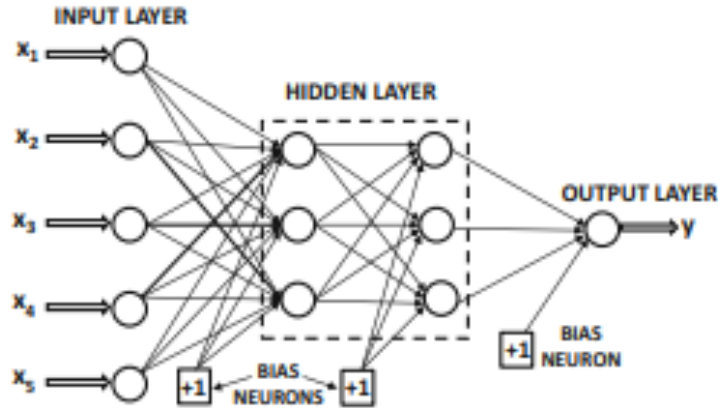
Dalam bentuk paling sederhananya, sebuah *neural network* hanya memiliki satu lapisan komputasi yang disebut *perceptron*. Pada arsitektur ini, terdapat n neuron *input* yang meneruskan fitur ke satu neuron *output* melalui sekumpulan bobot. Nilai *output* diperoleh dari penjumlahan bobot yang dikalikan dengan masing-masing nilai *input*, ditambah sebuah nilai *bias* b yang berfungsi menggeser ambang batas keputusan model agar lebih fleksibel dalam mempelajari pola data. Secara matematis, persamaan prediksi *output* dirumuskan sebagai:

$$z = \mathbf{w} \cdot \mathbf{x} + b = \sum_{i=1}^n w_i x_i + b \quad (2.1)$$

dengan $\mathbf{w}$ adalah vektor bobot, $\mathbf{x}$ adalah vektor input yang berisi nilai fitur-fitur data, dan b adalah bias.

Namun, *perceptron* hanya mampu memodelkan hubungan linear antara *input* dan *output*, sehingga tidak dapat menangani pola yang lebih kompleks. Untuk mengatasi keterbatasan ini, digunakan *multilayer neural network* yang memiliki satu atau lebih *hidden layer* di antara *input layer* dan *output layer*. Setiap lapisan menerima input dari lapisan sebelumnya dan meneruskannya searah menuju *output*

layer, dan pola aliran satu arah seperti ini dikenal sebagai *feed-forward*.



Gambar 2.3: Arsitektur *feed-forward neural network* (Aggarwal 2018)

Misalkan terdapat k *hidden layer* dengan masing-masing $p_1, \dots, p_k$ neuron, $\mathbf{x}$ sebagai input berisi n neuron, $\mathbf{h}_1, \dots, \mathbf{h}_k$ sebagai keluaran tiap *hidden layer* setelah melewati fungsi aktivasi, dan $\mathbf{o}$ sebagai output berisi m neuron. Matriks bobot $\mathbf{W}_1$ berdimensi $n \times p_1$ menghubungkan *input layer* dengan *hidden layer* pertama, $\mathbf{W}_r$ berdimensi $p_{r-1} \times p_r$ menghubungkan antar *hidden layer*, dan $\mathbf{W}_{k+1}$ berdimensi $p_k \times m$ menghubungkan *hidden layer* terakhir ke *output layer*. Proses komputasi pada tiap lapisan dirumuskan sebagai

$$\mathbf{h}_1 = \phi_1(\mathbf{W}_1^T \mathbf{x}) \quad (2.2)$$

$$\mathbf{h}_{r+1} = \phi_{r+1}(\mathbf{W}_{r+1}^T \mathbf{h}_r), \quad r \in \{1, \dots, k-1\} \quad (2.3)$$

$$\mathbf{o} = \phi_0(\mathbf{W}_{k+1}^T \mathbf{h}_k) \quad (2.4)$$

dengan ϕ_i adalah fungsi aktivasi yang diterapkan pada lapisan ke- i . Dengan demikian, output akhir $\mathbf{o}$ merupakan hasil transformasi bertingkat dari input $\mathbf{x}$ melalui seluruh lapisan jaringan secara berurutan.

2.3.1 Fungsi Aktivasi

Fungsi aktivasi merupakan komponen yang menentukan apakah dan seberapa besar sinyal dari sebuah neuron diteruskan ke lapisan berikutnya. Tanpa fungsi aktivasi, seluruh lapisan dalam jaringan hanya menghasilkan transformasi linear yang setara dengan satu lapisan tunggal, sehingga tidak mampu menangkap pola yang kompleks. Berikut adalah fungsi-fungsi aktivasi yang relevan dengan penelitian ini.

2.3.1.1 Sigmoid

Fungsi *sigmoid* memetakan nilai input ke rentang $(0, 1)$, sehingga cocok untuk interpretasi sebagai probabilitas 

$$\phi(v) = \frac{1}{1 + e^{-v}} \quad \text{} \quad (2.5)$$

dengan e adalah bilangan Euler, yaitu konstanta matematika bernilai sekitar 2,718. Ketika v bernilai sangat besar, output mendekati 1; ketika v sangat kecil (negatif), output mendekati 0. Meskipun intuitif, sigmoid rentan terhadap masalah *vanishing gradient*, yaitu kondisi di mana gradien menjadi sangat kecil sehingga pembaruan bobot pada lapisan awal hampir tidak terjadi, karena kurva sigmoid mendatar di kedua ujungnya.

2.3.1.2 Rectified Linear Unit (ReLU)

Rectified Linear Unit (ReLU) mengatasi masalah *vanishing gradient* dengan cara yang lebih sederhana: meneruskan nilai *input* apa adanya jika positif, dan mengembalikan nol jika negatif. 

$$\phi(v) = \max\{v, 0\} \quad \text{} \quad (2.6)$$

Sifat ini  membuat ReLU lebih efisien secara komputasi dan lebih mudah dilatih pada jaringan yang dalam, karena gradiennya selalu bernilai 1 untuk *input* positif sehingga sinyal pembaruan tidak melemah saat merambat mundur melalui banyak lapisan. 

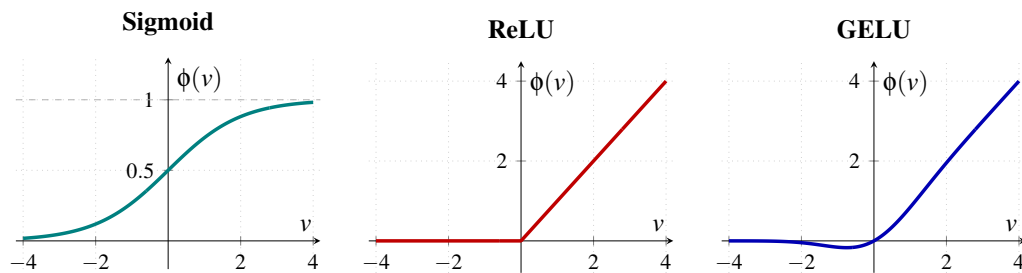
2.3.1.3 Gaussian Error Linear Unit (GELU)

Gaussian Error Linear Unit (GELU) (Hendrycks and Gimpel 2016) merupakan pengembangan dari ReLU yang digunakan pada arsitektur *Transformer* seperti BERT. Alih-alih  memotong nilai negatif secara langsung dengan batas keras di nol seperti ReLU, GELU menimbang setiap nilai *input* berdasarkan seberapa besar kemungkinannya bernilai positif secara probabilistik 

$$\phi(v) = v \cdot \Phi(v) = v \cdot \frac{1}{2} \left[1 + \operatorname{erf}\left(\frac{v}{\sqrt{2}}\right) \right] \quad (2.7)$$

dengan $\Phi(v)$ adalah fungsi distribusi kumulatif Gaussian standar, yaitu fungsi yang menyatakan peluang sebuah nilai acak bernilai lebih kecil dari v dalam distribusi normal. Simbol $\operatorname{erf}(\cdot)$ adalah fungsi error, yaitu bentuk integral dari distribusi Gaussian yang nilainya berkisar antara -1 dan 1 . Untuk nilai v yang besar dan

positif, GELU berperilaku mendekati ReLU, sedangkan di sekitar nol transisinya berlangsung mulus tanpa patahan.



Gambar 2.4: Grafik fungsi aktivasi: Sigmoid, ReLU, dan GELU

2.3.2 Fungsi Loss

Fungsi *loss* berperan sebagai ukuran seberapa jauh prediksi model menyimpang dari nilai yang sebenarnya. Nilai *loss* yang rendah menandakan prediksi model sudah mendekati target, sedangkan nilai yang tinggi menandakan sebaliknya. Untuk tugas klasifikasi multikelas, *categorical cross-entropy loss* merupakan pilihan yang umum digunakan.

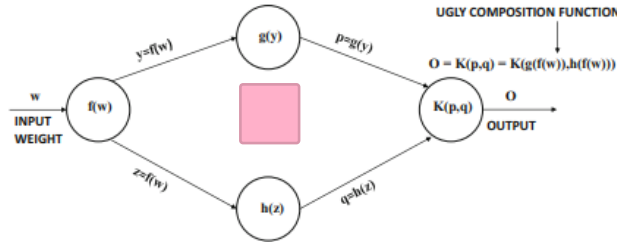
$$L = - \sum_{i=1}^n \sum_{j=1}^k y_{ij} \log(\hat{y}_{ij}) \quad (2.8)$$

dengan n adalah jumlah sampel, k adalah jumlah kelas, y_{ij} adalah label aktual dalam bentuk *one-hot encoded* bernilai 1 jika sampel ke- i termasuk kelas ke- j dan 0 jika tidak, serta $\hat{y}_{ij}$ adalah probabilitas prediksi model untuk sampel ke- i pada kelas ke- j (Aggarwal 2018; Goodfellow, Bengio, and Courville 2016). Semakin besar nilai L , semakin besar pula deviasi prediksi dari nilai aktual.

2.3.3 Backpropagation

Setelah *forward propagation* menghasilkan prediksi, model perlu mengetahui seberapa besar kontribusi setiap bobot terhadap kesalahan prediksi tersebut. Proses inilah yang dilakukan oleh *backpropagation*, yaitu algoritma yang menyebarkan informasi kesalahan dari *output layer* mundur ke *input layer* menggunakan aturan rantai turunan (*chain rule*). Secara garis besar, *backpropagation* mencakup tiga hal: (1) menghitung gradien fungsi *loss* terhadap setiap bobot menggunakan *chain rule*, (2) mendefinisikan sinyal kesalahan δ yang diwariskan dari satu lapisan ke lapisan sebelumnya, dan (3) menghitung δ tersebut secara efisien menggunakan teknik rekursi.

Aturan rantai memungkinkan kita menghitung bagaimana perubahan kecil pada suatu bobot di lapisan mana pun akan berdampak pada nilai *loss* akhir, meskipun antara bobot dan *loss* terdapat banyak lapisan komputasi di antaranya. Misalkan terdapat jaringan dengan *hidden layer* $h_1, h_2, \dots, h_k$ dan *output* o . Sebagai ilustrasi, pada jaringan sederhana dengan dua jalur menuju *output* seperti pada Gambar 2.5, gradien fungsi *loss* terhadap bobot w diturunkan dengan menelusuri seluruh jalur dari bobot tersebut menuju *output*:



Gambar 2.5: Struktur *multi-layer perceptron* pada jalur *input-output* (Aggarwal 2018)

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial o} \cdot \left(\frac{\partial o}{\partial p} \cdot \frac{\partial p}{\partial y} \cdot \frac{\partial y}{\partial w} + \frac{\partial o}{\partial q} \cdot \frac{\partial q}{\partial z} \cdot \frac{\partial z}{\partial w} \right) \quad (2.9)$$

dengan L adalah fungsi *loss*, o adalah nilai *output*, p dan q adalah nilai antara yang dihasilkan dari dua jalur berbeda menuju o , dan y, z adalah nilai pra-aktivasi pada masing-masing jalur tersebut. Setiap suku dalam tanda kurung merupakan satu jalur dari w ke o , dan karena ada dua jalur, keduanya dijumlahkan.

Pola penjumlahan atas semua jalur ini berlaku umum. Untuk jaringan dengan satu jalur dari h_r ke o , gradien terhadap bobot w_r yang menghubungkan lapisan ke- $(r-1)$ dan lapisan ke- r dapat ditulis sebagai:

$$\frac{\partial L}{\partial w_r} = \frac{\partial L}{\partial o} \cdot \left[\frac{\partial o}{\partial h_k} \prod_{i=r}^{k-1} \frac{\partial h_{i+1}}{\partial h_i} \right] \frac{\partial h_r}{\partial w_r}, \quad r \in \{1, \dots, k\} \quad (2.10)$$

Perkalian berantai $\prod_{i=r}^{k-1} \frac{\partial h_{i+1}}{\partial h_i}$ di sini merepresentasikan bagaimana kesalahan diteruskan dari lapisan h_r hingga mencapai *output* o , melewati satu per satu lapisan di antaranya. Untuk jaringan yang memiliki banyak jalur, misalkan $\mathcal{P}$ adalah himpunan semua jalur yang mungkin dari h_r ke o , maka persamaan di atas digeneralisasi menjadi:

$$\frac{\partial L}{\partial w_r} = \frac{\partial L}{\partial o} \cdot \left[\sum_{[h_r, \dots, h_k, o] \in \mathcal{P}} \frac{\partial o}{\partial h_k} \prod_{i=r}^{k-1} \frac{\partial h_{i+1}}{\partial h_i} \right] \frac{\partial h_r}{\partial w_r} \quad (2.11)$$

Perlu diperhatikan bahwa menghitung ekspresi dalam kurung siku secara langsung untuk setiap bobot sangat tidak efisien, karena jumlah jalur bisa tumbuh secara eksponensial seiring bertambahnya lapisan. Untuk mengatasi hal ini, bagian tersebut didefinisikan sebagai sinyal kesalahan δ_r , yaitu seberapa besar *loss* berubah akibat perubahan nilai pada lapisan ke- r :

$$\delta_r = \frac{\partial L}{\partial h_r} \quad (2.12)$$

sehingga persamaan (2.11) dapat ditulis lebih ringkas sebagai:

$$\frac{\partial L}{\partial w_r} = \delta_r \cdot \frac{\partial h_r}{\partial w_r} = \delta_r \cdot f'(z_r) \cdot h_{r-1} \quad (2.13)$$

dengan $f'(z_r)$ adalah turunan fungsi aktivasi pada lapisan ke- r yang dievaluasi pada nilai pra-aktivasi $z_r = \mathbf{w}_r^T \mathbf{h}_{r-1}$, dan h_{r-1} adalah keluaran lapisan sebelumnya yang menjadi input lapisan ke- r . Dengan demikian, untuk memperbarui setiap bobot, yang diperlukan hanyalah nilai δ_r pada lapisan yang bersangkutan.

Nilai δ_r sendiri tidak dihitung dari nol untuk setiap lapisan, melainkan diwariskan secara rekursif dari lapisan yang lebih dekat ke *output*. Titik awal rekursi ini adalah lapisan output itu sendiri:

$$\delta_o = \frac{\partial L}{\partial o} \quad (2.14)$$

Kemudian, δ_r untuk setiap lapisan tersembunyi dihitung dari δ_{r+1} lapisan di atasnya, dengan menjumlahkan kontribusi dari semua neuron pada lapisan ke- $(r+1)$:

$$\delta_r = \frac{\partial L}{\partial h_r} = \sum_{h_{r+1} \in \mathbf{h}_{r+1}} \frac{\partial L}{\partial h_{r+1}} \cdot \frac{\partial h_{r+1}}{\partial h_r} = \sum_{h_{r+1} \in \mathbf{h}_{r+1}} \delta_{r+1} \cdot \frac{\partial h_{r+1}}{\partial h_r} \quad (2.15)$$

dengan $\mathbf{h}_{r+1}$ adalah vektor yang berisi seluruh neuron pada lapisan ke- $(r+1)$, dan penjumlahan dilakukan karena satu neuron di lapisan ke- r dapat terhubung ke banyak neuron di lapisan ke- $(r+1)$. Suku $\frac{\partial h_{r+1}}{\partial h_r}$ dapat dijabarkan lebih lanjut menggunakan aturan rantai:

$$\frac{\partial h_{r+1}}{\partial h_r} = \frac{\partial h_{r+1}}{\partial z_{r+1}} \cdot \frac{\partial z_{r+1}}{\partial h_r} = f'(z_{r+1}) \cdot w_{r+1} \quad (2.16)$$

dengan z_{r+1} adalah nilai pra-aktivasi pada lapisan ke- $(r+1)$, $f'(z_{r+1})$ adalah turunan fungsi aktivasinya, dan w_{r+1} adalah bobot yang menghubungkan neuron di lapisan ke- r dengan neuron di lapisan ke- $(r+1)$. Substitusikan ke persamaan (2.15), diperoleh bentuk akhir rekursi δ :

$$\delta_r = \sum_{h_{r+1} \in \mathbf{h}_{r+1}} f'(z_{r+1}) \cdot w_{r+1} \cdot \delta_{r+1} \quad (2.17)$$

Dengan menggunakan rekursi ini, nilai δ dihitung mulai dari lapisan *output*, kemudian diwariskan satu per satu ke lapisan-lapisan sebelumnya — itulah mengapa proses ini disebut *backpropagation*, karena sinyal kesalahan merambat mundur melalui jaringan. Gradien yang diperoleh dari proses ini kemudian digunakan untuk memperbarui bobot melalui teknik optimasi.

2.3.4 Teknik Optimasi

Teknik optimasi memanfaatkan gradien fungsi *loss* yang dihasilkan oleh *backpropagation* untuk memperbarui parameter model secara iteratif dengan tujuan meminimalkan kesalahan prediksi. Dalam praktiknya, pembaruan tidak dilakukan atas seluruh data sekaligus, melainkan secara bertahap dalam kelompok kecil yang disebut *batch*. Satu putaran lengkap di mana seluruh data pelatihan telah diproses disebut *epoch*.

2.3.4.1 Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD) merupakan metode optimasi paling mendasar yang menjadi fondasi sebagian besar algoritma pelatihan *neural network*. Ide utamanya sederhana: setiap bobot diperbarui dengan bergerak berlawanan arah dari gradien, karena arah gradien menunjuk ke arah naiknya nilai *loss*, sehingga bergerak berlawanan arah berarti mendekati nilai *loss* yang lebih kecil:

$$\mathbf{w}^{[t+1]} = \mathbf{w}^{[t]} - \eta \nabla L(\mathbf{w})^{[t]} \quad (2.18)$$

dengan $\mathbf{w}^{[t]}$ adalah vektor bobot pada iterasi ke- t , $\nabla L(\mathbf{w})^{[t]}$ adalah gradien fungsi *loss* terhadap bobot pada iterasi tersebut, dan η adalah *learning rate*, yaitu hiperparameter yang mengontrol seberapa besar langkah pembaruan yang diambil. Nilai η yang terlalu kecil membuat proses konvergensi sangat lambat, sedangkan nilai yang terlalu besar berisiko membuat model melompat melewati titik optimal dan tidak stabil.

2.3.4.2 Adaptive Moment Estimation (Adam)

Adaptive Moment Estimation (Adam) (Kingma and Ba 2015) merupakan penyempurnaan dari SGD yang mengadaptasi besar langkah pembaruan secara otomatis untuk setiap parameter. Jika SGD menggunakan satu *learning rate* yang sama untuk

semua bobot, Adam menyesuaikan berdasarkan riwayat gradien masing-masing bobot. Adam melacak dua besaran sekaligus: rata-rata gradien yang disebut momen pertama $\mathbf{v}^{[t]}$, dan rata-rata kuadrat gradien yang disebut momen kedua $\mathbf{s}^{[t]}$:

$$\mathbf{v}^{[t]} = \beta_1 \mathbf{v}^{[t-1]} + (1 - \beta_1) \nabla L(\mathbf{w})^{[t]} \quad (2.19)$$

$$\mathbf{s}^{[t]} = \beta_2 \mathbf{s}^{[t-1]} + (1 - \beta_2) \left(\nabla L(\mathbf{w})^{[t]} \right)^2 \quad (2.20)$$

dengan β_1 dan β_2 adalah koefisien peluruhan yang mengontrol seberapa besar pengaruh gradien lama terhadap estimasi saat ini. Karena kedua momen diinisialisasi dengan nol, estimasinya cenderung bias ke nol pada iterasi-iterasi awal. Untuk mengoreksi bias ini, dilakukan normalisasi terhadap masing-masing momen:

$$\hat{\mathbf{v}}^{[t]} = \frac{\mathbf{v}^{[t]}}{1 - \beta_1^t}, \quad \hat{\mathbf{s}}^{[t]} = \frac{\mathbf{s}^{[t]}}{1 - \beta_2^t} \quad (2.21)$$

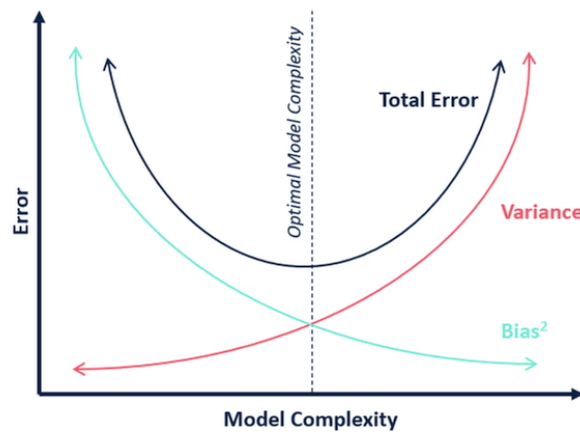
Semakin besar t , nilai β_1^t dan β_2^t mendekati nol sehingga koreksi ini semakin tidak berpengaruh, artinya koreksi hanya signifikan di awal pelatihan. Pembaruan bobot kemudian dilakukan menggunakan kedua momen yang telah dikoreksi:

$$\mathbf{w}^{[t+1]} = \mathbf{w}^{[t]} - \frac{\eta}{\sqrt{\hat{\mathbf{s}}^{[t]} + \epsilon}} \hat{\mathbf{v}}^{[t]} \quad (2.22)$$

dengan $\epsilon = 10^{-8}$ adalah nilai kecil yang ditambahkan untuk mencegah pembagian dengan nol (A. Zhang *et al.* 2023). Pembagian dengan $\sqrt{\hat{\mathbf{s}}^{[t]}}$ membuat langkah pembaruan otomatis mengecil untuk parameter yang gradinya besar dan berfluktuasi, serta membesar untuk parameter yang gradiennya kecil dan konsisten.

2.3.5 Regularisasi

Dalam melatih model, terdapat dua fenomena yang perlu dihindari. *Underfitting* terjadi ketika model gagal mempelajari pola data pelatihan dengan baik, ditandai dengan nilai *loss* yang tinggi bahkan pada data pelatihan itu sendiri. Sebaliknya, *overfitting* terjadi ketika model terlalu menyesuaikan diri dengan data pelatihan sehingga performanya menurun drastis saat diuji pada data baru yang belum pernah dilihat sebelumnya.



Gambar 2.6: Ilustrasi fenomena *underfitting* dan *overfitting*

Untuk mencegah *overfitting*, salah satu teknik yang umum digunakan adalah *weight decay* atau L_2 *regularization*. Ide dasarnya adalah memberikan penalti tambahan pada fungsi *loss* setiap kali bobot model bernilai terlalu besar, sehingga model terdorong untuk tetap sederhana:

$$\tilde{L}(\mathbf{w}) = L(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|^2 \quad (2.23)$$

dengan $L(\mathbf{w})$ adalah fungsi *loss* asal, $\|\mathbf{w}\|^2$ adalah jumlah kuadrat seluruh bobot dalam model, dan λ adalah parameter yang mengontrol seberapa kuat penalti tersebut diterapkan. Semakin besar λ , semakin kuat model didorong untuk menggunakan bobot-bobot yang kecil.

2.3.6 Metrik Evaluasi

Setelah model dilatih, performa klasifikasinya diukur menggunakan data yang belum pernah dilihat selama pelatihan. Titik awal evaluasi adalah *confusion matrix*, yaitu tabel yang membandingkan prediksi model terhadap label aktual. Dari tabel ini diperoleh empat komponen dasar: *True Positive* (TP) yaitu sampel positif yang diprediksi benar, *True Negative* (TN) yaitu sampel negatif yang diprediksi benar, *False Positive* (FP) yaitu sampel negatif yang salah diprediksi sebagai positif, dan *False Negative* (FN) yaitu sampel positif yang salah diprediksi sebagai negatif.

Dari keempat komponen tersebut, diturunkan empat metrik utama. Akurasi mengukur proporsi keseluruhan prediksi yang benar:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.24)$$

Precision mengukur seberapa tepat model ketika menyatakan sesuatu sebagai

positif, artinya dari semua yang diprediksi positif, berapa banyak yang benar-benar positif:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2.25)$$

Recall mengukur seberapa lengkap model dalam menangkap semua sampel positif yang ada, artinya dari semua yang sebenarnya positif, berapa banyak yang berhasil ditemukan:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2.26)$$

Kedua metrik ini sering kali saling bertentangan: model yang terlalu berhati-hati cenderung memiliki *precision* tinggi namun *recall* rendah, dan sebaliknya. *F1-Score* menyeimbangkan keduanya dalam satu nilai tunggal menggunakan rata-rata harmonik:

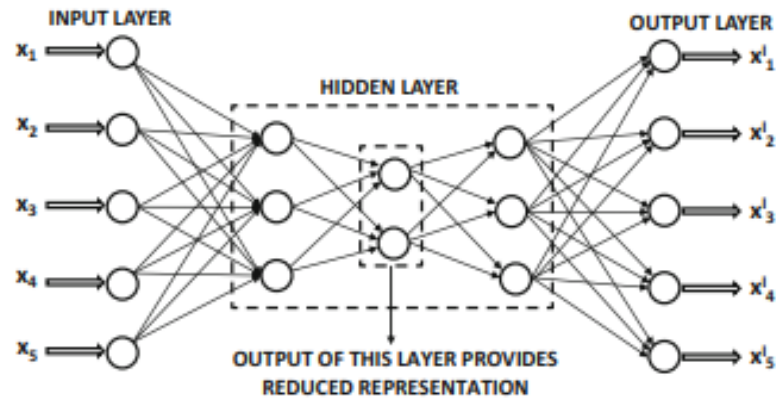
$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times TP}{2 \times TP + FP + FN} \quad (2.27)$$

F1-Score sangat berguna pada dataset yang tidak seimbang karena tidak memberikan bobot berlebihan pada kelas yang mendominasi.

2.4 Autoencoder

Pemahaman tentang arsitektur *feed-forward neural network* di atas menjadi landasan untuk memahami Autoencoder, yang pada dasarnya adalah sebuah jaringan saraf yang dilatih bukan untuk memprediksi label, melainkan untuk merekonstruksi inputnya kembali (Goodfellow, Bengio, and Courville 2016). Karena tidak memerlukan label, Autoencoder termasuk ke dalam paradigma *unsupervised learning* (Hinton and Salakhutdinov 2006).

Arsitektur Autoencoder terdiri dari dua bagian yang bekerja secara berkesinambungan (Vincent *et al.* 2010). Bagian pertama adalah *encoder*, yang bertugas memampatkan representasi input dari dimensi tinggi ke dimensi yang lebih rendah. Bagian kedua adalah *decoder*, yang bertugas merekonstruksi kembali representasi tersebut ke dimensi semula. Titik terkecil di antara keduanya, tempat representasi paling ringkas berada, disebut *bottleneck* atau ruang laten.



Gambar 2.7: Arsitektur Autoencoder menunjukkan proses *encoding* dan *decoding* (Aggarwal 2018)

2.4.1 Encoder dan Decoder

Secara formal, misalkan $\mathbf{x} \in \mathbb{R}^D$ adalah vektor input berdimensi D . *Encoder* memetakan $\mathbf{x}$ ke dalam vektor laten $\mathbf{z} \in \mathbb{R}^d$ dengan $d < D$, artinya representasi yang dihasilkan memiliki dimensi yang lebih kecil dari aslinya.

$$\mathbf{z} = f_{\theta}(\mathbf{x}) = \sigma_h(\mathbf{W}_h \mathbf{x} + \mathbf{b}_h) \quad (2.28)$$

dengan $\mathbf{W}_h \in \mathbb{R}^{d \times D}$ adalah matriks bobot *encoder*, $\mathbf{b}_h \in \mathbb{R}^d$ adalah vektor bias, dan $\sigma_h(\cdot)$ adalah fungsi aktivasi seperti ReLU atau sigmoid. Simbol θ merepresentasikan seluruh parameter (bobot dan bias) pada bagian *encoder*.

Selanjutnya, *decoder* memetakan kembali vektor laten $\mathbf{z}$ ke dalam ruang dimensi asal untuk menghasilkan rekonstruksi $\hat{\mathbf{x}} \in \mathbb{R}^D$:

$$\hat{\mathbf{x}} = g_{\theta'}(\mathbf{z}) = \sigma_o(\mathbf{W}_o \mathbf{z} + \mathbf{b}_o) \quad (2.29)$$

dengan $\mathbf{W}_o \in \mathbb{R}^{D \times d}$ adalah matriks bobot *decoder*, $\mathbf{b}_o \in \mathbb{R}^D$ adalah vektor bias, dan $\sigma_o(\cdot)$ adalah fungsi aktivasi output. Simbol θ' merepresentasikan seluruh parameter pada bagian *decoder*. Secara keseluruhan, Autoencoder melakukan transformasi

$$\hat{\mathbf{x}} = g_{\theta'}(f_{\theta}(\mathbf{x})) \quad (2.30)$$

2.4.2 Fungsi Objektif

Autoencoder dilatih dengan meminimalkan selisih antara *input* asal dan hasil rekonstruksinya. Pada penelitian ini, fungsi *loss* yang digunakan adalah *Mean Squared*

Error (MSE), yaitu rata-rata kuadrat selisih antara setiap elemen *input* dan rekonstruksinya. Semakin kecil nilai MSE, semakin dekat rekonstruksi dengan *input* aslinya:

$$\mathcal{L}(\theta, \theta') = \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i - g_{\theta'}(f_{\theta}(\mathbf{x}_i))\|^2 \quad (2.31)$$

dengan n adalah jumlah sampel dalam *dataset* dan $\|\cdot\|^2$ adalah kuadrat norma Euclidean, yang secara eksplisit berarti menjumlahkan kuadrat selisih tiap dimensi antara *input* $\mathbf{x}_i$ dan rekonstruksinya $\hat{\mathbf{x}}_i$. Parameter *encoder* θ dan *decoder* θ' diperbarui secara bersama-sama menggunakan *backpropagation* dan optimasi Adam untuk meminimalkan nilai *loss* ini (Vincent *et al.* 2010).

2.5 Bidirectional Encoder Representations from Transformers (BERT)

Bidirectional Encoder Representations from Transformers (BERT) merupakan model bahasa yang diperkenalkan oleh Devlin *et al.* (2018) yang merevolusi bidang *natural language processing* (NLP). Berbeda dari model bahasa sebelumnya yang membaca teks secara satu arah, BERT memahami makna suatu kata dengan mempertimbangkan konteks dari kiri maupun kanan secara bersamaan. Kemampuan ini dimungkinkan oleh arsitekturnya yang didasarkan pada *Transformer Encoder*, yaitu sebuah mekanisme yang memungkinkan setiap kata dalam kalimat untuk saling memperhatikan satu sama lain secara paralel.

Sebelum digunakan, BERT terlebih dahulu dilatih pada korpus teks berskala sangat besar, yakni Wikipedia bahasa Inggris dan BookCorpus, sehingga bobotnya sudah mengandung pemahaman bahasa yang kaya. Pada penelitian ini, bobot tersebut tidak diubah sama sekali. BERT digunakan sepenuhnya sebagai ekstraktor fitur yang dibekukan (*frozen*), artinya teks ulasan dimasukkan ke dalam BERT dan representasi yang dihasilkannya langsung digunakan sebagai fitur untuk tahap berikutnya.

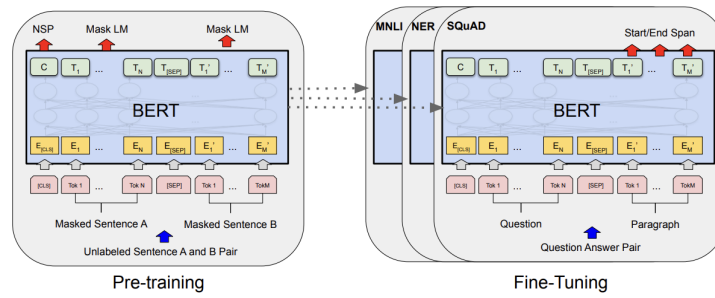
2.5.1 Arsitektur BERT

Arsitektur BERT tersusun dari L lapisan *Transformer Encoder* yang identik, masing-masing terdiri dari dua sub-lapisan: *Multi-Head Self-Attention* dan *Position-wise Feed-Forward Network*. Keluaran setiap lapisan dihitung sebagai:

$$\mathbf{H}^{(\ell)} = \text{LayerNorm}\left(\mathbf{H}^{(\ell-1)} + \text{MultiHeadAttention}(\mathbf{H}^{(\ell-1)})\right) \quad (2.32)$$

$$\mathbf{H}_{\text{out}}^{(\ell)} = \text{LayerNorm}\left(\mathbf{H}^{(\ell)} + \text{FFN}(\mathbf{H}^{(\ell)})\right) \quad (2.33)$$

dengan $\mathbf{H}^{(\ell)}$ adalah representasi tersembunyi (*hidden state*) pada lapisan ke- ℓ , dan LayerNorm adalah normalisasi lapisan yang membantu stabilitas pelatihan. Jumlah lapisan L , dimensi *hidden state* H , dan jumlah *attention head* A bergantung pada varian model *pre-trained* yang dipilih.



Gambar 2.8: Arsitektur BERT dengan beberapa lapisan *Transformer Encoder* Devlin et al. 2018

2.5.2 Mekanisme *Self-Attention*

Inti dari arsitektur BERT adalah mekanisme *self-attention*, yaitu cara model menentukan seberapa besar perhatian yang perlu diberikan oleh setiap kata terhadap kata-kata lain dalam kalimat yang sama. Diberikan urutan *input* $\mathbf{X} \in \mathbb{R}^{n \times d}$ dengan n token dan dimensi d , setiap token diproyeksikan ke dalam tiga representasi berbeda melalui matriks bobot yang dapat dipelajari, yaitu *query* $\mathbf{Q}$, *key* $\mathbf{K}$, dan *value* $\mathbf{V}$:

$$\mathbf{Q} = \mathbf{XW}_Q \in \mathbb{R}^{n \times d_k} \quad (2.34)$$

$$\mathbf{K} = \mathbf{XW}_K \in \mathbb{R}^{n \times d_k} \quad (2.35)$$

$$\mathbf{V} = \mathbf{XW}_V \in \mathbb{R}^{n \times d_v} \quad (2.36)$$

dengan $\mathbf{W}_Q, \mathbf{W}_K \in \mathbb{R}^{d \times d_k}$ dan $\mathbf{W}_V \in \mathbb{R}^{d \times d_v}$ adalah matriks bobot yang dapat dipelajari. Secara intuitif, *query* mewakili kata yang sedang mencari konteks, *key* mewakili kata-kata yang bisa menjadi konteks, dan *value* adalah informasi yang akan diambil jika konteks tersebut dianggap relevan.

Skor relevansi antara setiap pasangan kata dihitung melalui perkalian *query* dan *key*, lalu dinormalisasi dengan $\sqrt{d_k}$ untuk menjaga skala nilai agar gradien tidak

menjadi terlalu kecil. Hasilnya kemudian diubah menjadi distribusi probabilitas menggunakan *softmax*, sehingga bobot perhatian seluruh kata selalu berjumlah satu: 

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V} \quad (2.37)$$

Untuk meningkatkan kemampuan representasi, BERT menjalankan mekanisme *attention* ini secara paralel sebanyak h kali pada *subspace* yang berbeda, yang disebut *Multi-Head Attention*. Setiap hasil digabungkan dan diproyeksikan kembali ke dimensi semula:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}_O \quad (2.38)$$

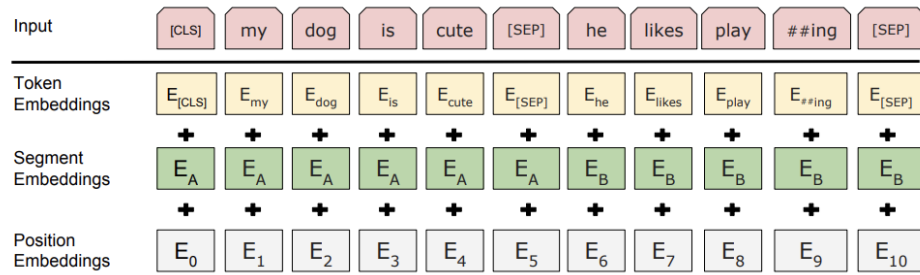
dengan $\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_Q^i, \mathbf{K}\mathbf{W}_K^i, \mathbf{V}\mathbf{W}_V^i)$. Pada BERT-Base, $h = 12$ dan $d_k = d_v = 64$.

2.5.3 Representasi Input BERT

Sebelum teks dapat diproses oleh BERT, setiap token perlu diubah menjadi vektor numerik. BERT merepresentasikan setiap token sebagai penjumlahan dari tiga jenis *embedding* yang masing-masing membawa informasi berbeda:

$$\mathbf{E}_i = \mathbf{E}_{\text{token}}^{(i)} + \mathbf{E}_{\text{segment}}^{(i)} + \mathbf{E}_{\text{position}}^{(i)} \quad (2.39)$$

Token embedding merepresentasikan makna dari kata atau *subword* itu sendiri. BERT menggunakan tokenisasi WordPiece dengan kosakata sebesar 30.000 token, sehingga kata yang tidak dikenal sekalipun dapat dipecah menjadi potongan-potongan yang lebih kecil dan tetap dapat direpresentasikan. *Segment embedding* membedakan asal kalimat dari setiap token, berguna ketika *input* terdiri dari dua kalimat sekaligus. *Position embedding* menyandikan posisi setiap token dalam urutan, karena mekanisme *self-attention* sendiri tidak memiliki informasi urutan secara bawaan. Berbeda dari Transformer asli yang menggunakan rumus sinusoidal, BERT mempelajari *position embedding* ini langsung dari data dengan panjang urutan maksimum 512 token.



Gambar 2.9: Representasi *input* BERT yang terdiri dari *token*, *segment*, dan *position embedding* Devlin *et al.* 2018

Selain token biasa, BERT menyisipkan token khusus [CLS] di awal setiap urutan *input*. Token ini tidak mewakili kata tertentu, melainkan dirancang untuk mengakumulasi informasi dari seluruh token dalam kalimat melalui lapisan-lapisan *self-attention*. Pada penelitian ini, representasi token [CLS] pada lapisan terakhir BERT inilah yang digunakan sebagai vektor fitur untuk setiap ulasan.

2.5.4 Ekstraksi Fitur dengan BERT

Pada penelitian ini, BERT digunakan sepenuhnya sebagai ekstraktor fitur dengan bobot yang dibekukan. Setiap ulasan teks dimasukkan ke dalam BERT dengan panjang maksimum 128 token, kemudian representasi *hidden state* pada posisi token [CLS] dari lapisan terakhir diambil sebagai fitur:

$$\mathbf{f} = \mathbf{H}_{[CLS]}^{(L)} \in \mathbb{R}^d \quad (2.40)$$

dengan $\mathbf{H}_{[CLS]}^{(L)}$ adalah *hidden state* token [CLS] pada lapisan ke- L (lapisan terakhir), dan d adalah dimensi *hidden state* yang bergantung pada varian model *pre-trained* yang digunakan. Vektor $\mathbf{f}$ inilah yang selanjutnya menjadi representasi numerik dari setiap ulasan dan digunakan sebagai *input* untuk Autoencoder pada tahap berikutnya.

2.6 Multi-Task Learning

Multi-Task Learning (MTL) merupakan paradigma dalam *machine learning* yang bertujuan meningkatkan kemampuan generalisasi model dengan memanfaatkan informasi dari beberapa tugas yang saling berkaitan secara bersamaan (Caruana 1997). Ide dasarnya sederhana, ketika dua tugas memiliki keterkaitan, sinyal pembelajaran dari satu tugas dapat membantu tugas lainnya belajar lebih baik karena keduanya berbagi pola atau struktur yang sama dalam data.

Konsep kunci yang mendasari MTL adalah *inductive transfer*, yaitu kemampuan model untuk memindahkan pengetahuan yang dipelajari dari satu tugas ke tugas lainnya. Caruana (1997) menjelaskan bahwa MTL mencapai hal ini dengan cara mempelajari beberapa tugas secara paralel menggunakan representasi bersama, sehingga apa yang dipelajari untuk satu tugas turut membantu tugas lain untuk dipelajari dengan lebih baik. Dalam konteks ini, tugas-tugas lain berfungsi sebagai *inductive bias*, yaitu tekanan yang mendorong model untuk lebih menyukai hipotesis yang mampu menjelaskan lebih dari satu tugas sekaligus (Ruder 2017). Analoginya seperti seseorang yang belajar dua bahasa serumpun secara bersamaan: pemahaman terhadap satu bahasa mempercepat penguasaan bahasa yang lain karena keduanya berbagi banyak pola tata bahasa yang sama.

Pada penelitian ini, MTL digunakan untuk mempelajari analisis sentimen dan pendeteksian topik secara bersamaan. Kedua tugas ini memiliki keterkaitan yang kuat karena sama-sama bergantung pada pemahaman semantis teks ulasan, sehingga representasi yang berguna untuk mendeteksi topik juga cenderung berguna untuk mengklasifikasikan sentimen, dan sebaliknya.

2.6.1 Formulasi MTL

Misalkan terdapat T tugas yang dipelajari secara bersamaan. Untuk setiap tugas $t \in \{1, 2, \dots, T\}$, terdapat dataset $\mathcal{D}_t = \{(\mathbf{x}_i^{(t)}, y_i^{(t)})\}_{i=1}^{n_t}$ dan fungsi *loss* $\mathcal{L}_t$ yang mengukur kesalahan prediksi pada tugas tersebut. Tujuan MTL adalah meminimalkan jumlah tertimbang dari *loss* semua tugas secara **bersamaan**:

$$\mathcal{L}_{\text{MTL}} = \sum_{t=1}^T \alpha_t \mathcal{L}_t(\theta_t, \theta_{\text{shared}}) \quad (2.41)$$

dengan α_t adalah bobot yang mengontrol seberapa besar kontribusi tugas ke- t terhadap total *loss*, θ_t adalah parameter yang spesifik untuk tugas ke- t , dan θ_{shared} adalah parameter yang dibagikan bersama oleh semua tugas. Parameter optimal diperoleh dengan meminimalkan persamaan tersebut secara **bersama-sama**:

$$\theta_{\text{shared}}^*, \{\theta_t^*\}_{t=1}^T = \arg \min_{\theta_{\text{shared}}, \{\theta_t\}} \mathcal{L}_{\text{MTL}} \quad (2.42)$$

Pada penelitian ini, fungsi *loss* total terdiri dari dua komponen aktif yang masing-masing membawa bobot tersendiri. Meskipun secara umum formulasi MTL dapat menyertakan *reconstruction loss* dari Autoencoder, pada penelitian ini komponen tersebut tidak digunakan karena bobotnya ditetapkan nol. Komponen pertama yang aktif adalah *clustering loss* yang menggunakan *Kullback-Leibler Diver-*

gence (KLD) untuk mendekatkan distribusi penugasan kluster Q ke distribusi target P yang lebih tajam, sehingga mendorong kluster yang semakin kohesif dari iterasi ke iterasi. Komponen kedua adalah *sentiment loss* menggunakan *categorical cross-entropy* untuk tugas klasifikasi sentimen. Keduanya digabungkan sebagai berikut:

$$\mathcal{L} = \gamma \cdot \mathcal{L}_{\text{cluster}} + \eta \cdot \mathcal{L}_{\text{sentiment}} \quad (2.43)$$

dengan γ dan η adalah hiperparameter yang mengontrol bobot masing-masing komponen. *Clustering loss* diformulasikan sebagai KLD antara distribusi target P dan distribusi penugasan Q :

$$\mathcal{L}_{\text{cluster}} = D_{KL}(P||Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}} \quad (2.44)$$

dengan q_{ij} adalah probabilitas sampel ke- i termasuk ke kluster ke- j , dan p_{ij} adalah distribusi target yang diperoleh dengan mempertajam Q agar sampel yang sudah yakin masuk ke suatu kluster semakin diyakini penugasannya:

$$p_{ij} = \frac{q_{ij}^2 / \sum_i q_{ij}}{\sum_{j'} q_{ij'}^2 / \sum_i q_{ij'}} \quad (2.45)$$

Pembagian dengan $\sum_i q_{ij}$ berfungsi menormalisasi frekuensi kluster agar kluster yang besar tidak mendominasi, sedangkan operasi kuadrat pada q_{ij} mempertajam distribusi sehingga sampel yang ragu-ragu akan didorong untuk lebih memilih satu kluster. Semakin besar nilai $D_{KL}(P||Q)$, semakin jauh distribusi penugasan kluster dari kondisi idealnya, sehingga model terdorong untuk mempertegas keputusan klasterisasinya.

2.6.2 Hard Parameter Sharing

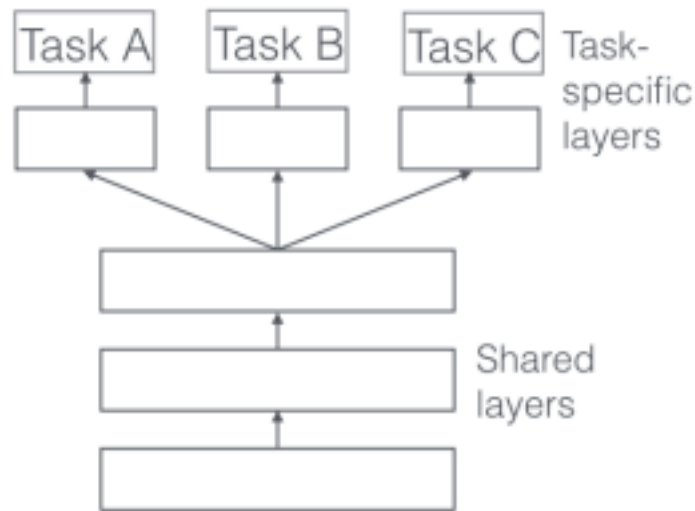
Hard parameter sharing merupakan pendekatan MTL yang paling umum digunakan dalam *deep learning* (Ruder 2017). Pada pendekatan ini, lapisan-lapisan tersembunyi dibagikan secara langsung oleh semua tugas, sementara setiap tugas memiliki lapisan *output* tersendiri yang bersifat spesifik. Secara formal, untuk *input* $\mathbf{x}$, proses komputasinya adalah:

$$\mathbf{h}_{\text{shared}} = f_{\text{shared}}(\mathbf{x}; \theta_{\text{shared}}) \quad (2.46)$$

$$\hat{y}_t = f_t(\mathbf{h}_{\text{shared}}; \theta_t), \quad t = 1, \dots, T \quad (2.47)$$

dengan f_{shared} adalah jaringan bersama yang menghasilkan representasi $\mathbf{h}_{\text{shared}}$,

dan f_t adalah jaringan spesifik tugas ke- t yang mengambil representasi **tersebut** sebagai *input*. Karena gradien dari semua tugas mengalir ke lapisan bersama selama *backpropagation*, representasi yang terbentuk terdorong untuk menangkap pola yang berguna bagi semua tugas sekaligus. Baxter (2000) menunjukkan secara teoritis bahwa berbagi parameter antar tugas secara langsung mengurangi risiko *overfitting* karena model harus menemukan solusi yang memuaskan banyak tugas sekaligus, bukan hanya menghafalkan pola dari satu tugas saja.



Gambar 2.1 Ilustrasi arsitektur *hard parameter sharing* dalam *Multi-Task Learning* (Ruder 2017)

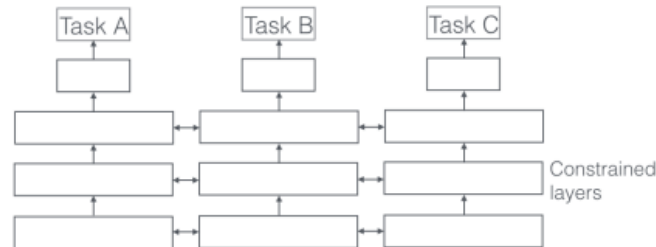
Pada penelitian ini, pendekatan *hard parameter sharing* diterapkan dengan menjadikan Autoencoder sebagai jaringan bersama. Representasi laten $\mathbf{z}$ yang dihasilkan oleh *encoder* digunakan secara bersamaan oleh dua *head* yang berbeda: satu untuk pendeteksian topik melalui *clustering layer*, dan satu lagi untuk klasifikasi sentimen melalui jaringan *sentiment classifier*.

2.6.3 Soft Parameter Sharing

Soft parameter sharing mengambil pendekatan yang berbeda. Setiap tugas memiliki model tersendiri dengan parameter masing-masing, namun parameter antar model diatur agar tidak terlalu jauh berbeda melalui regularisasi (Ruder 2017). Salah satu cara yang umum digunakan adalah menambahkan penalti ℓ_2 terhadap perbedaan parameter antar tugas ke dalam fungsi *loss*:

$$\mathcal{L}_{\text{soft}} = \sum_{t=1}^T \mathcal{L}_t(\theta_t) + \lambda \sum_{t=1}^T \sum_{t'=t+1}^T \|\theta_t - \theta_{t'}\|_2^2 \quad (2.48)$$

dengan λ adalah koefisien regularisasi yang mengontrol seberapa mirip parameter antar tugas harus dijaga, dan $\|\theta_t - \theta_{t'}\|_2^2$ adalah kuadrat norma Euclidean dari selisih parameter tugas ke- t dan tugas ke- t' , yang mengukur seberapa jauh kedua set parameter tersebut berbeda. Semakin besar λ , semakin kuat tekanan agar parameter antar tugas tetap berdekatan.



Gambar 2.1 Ilustrasi arsitektur *soft parameter sharing*. Setiap tugas memiliki model tersendiri, namun parameter antar model diregularisasi agar tetap serupa (Ruder 2017)

Perbedaan mendasar antara keduanya terletak pada bagaimana pengetahuan dibagikan. Pada *hard parameter sharing*, pembagian terjadi secara eksplisit melalui bobot yang benar-benar identik sehingga semua tugas menggunakan representasi yang persis sama. Pada *soft parameter sharing*, setiap tugas tetap memiliki ruang belajar sendiri namun didorong untuk tidak terlalu menyimpang satu sama lain, sehingga lebih fleksibel namun membutuhkan lebih banyak parameter dan pengaturan λ yang tepat.

2.7 Clustering

Clustering merupakan teknik fundamental dalam *unsupervised learning* yang bertujuan mengelompokkan data ke dalam kelompok-kelompok berdasarkan tingkat kemiripan, tanpa memerlukan label yang diberikan sebelumnya (Bezdek 1984). Salah satu aspek yang membedakan berbagai metode *clustering* adalah bagaimana keanggotaan suatu titik data terhadap kluster didefinisikan. *Hard clustering* membatasi setiap titik data hanya menjadi anggota satu kluster tertentu, menghasilkan partisi yang tegas dan saling eksklusif. Sebaliknya, *soft clustering* memungkinkan satu titik data memiliki derajat keanggotaan ke beberapa kluster sekaligus, sehingga lebih fleksibel untuk data yang ambigu atau memiliki tumpang tindih antar kluster.

Metode *clustering* dapat diklasifikasikan ke dalam beberapa pendekatan, di antaranya *centroid-based*, *distribution-based*, *connectivity-based*, dan *density-based*. Penelitian ini menggunakan pendekatan *centroid-based clustering*, yaitu metode yang memisahkan data berdasarkan jarak setiap titik data terhadap pusat kluster

atau *centroid* (Yusdiansyah and Suryadi 2019). Untuk mengukur jarak tersebut, digunakan jarak Euclidean yang didefinisikan sebagai:

$$\|\mathbf{p} - \mathbf{q}\| = \sqrt{\sum_{d=1}^D (p_d - q_d)^2} \quad (2.49)$$

dengan $\mathbf{p} = (p_1, \dots, p_D)$ dan $\mathbf{q} = (q_1, \dots, q_D)$ adalah dua titik dalam ruang berdimensi D . Semakin kecil nilai jarak ini, semakin mirip kedua titik tersebut.

Dalam penelitian ini, *clustering* dimanfaatkan sebagai metode pendeteksian topik menggunakan pendekatan *Deep Embedded Clustering* (DEC) (Xie, Girshick, and Farhadi 2016), yang melakukan pembelajaran representasi laten dan *clustering* secara simultan. DEC menggunakan *soft clustering* melalui distribusi probabilitas penugasan kluster, sehingga setiap ulasan tidak sekadar ditetapkan ke satu topik, melainkan memiliki derajat keanggotaan terhadap setiap topik yang ada. Sebagai tahap inisialisasi, DEC menggunakan algoritma K-Means untuk menentukan pusat kluster awal di ruang *embedding*.

2.7.1 K-Means Clustering

K-Means merupakan algoritma *centroid-based clustering* yang paling banyak digunakan. Algoritma ini mengelompokkan data dengan cara meminimalkan total jarak antara setiap titik data dan pusat klasternya. K-Means termasuk ke dalam kategori *hard clustering*, di mana setiap titik data hanya dapat menjadi anggota satu kluster saja pada setiap iterasi.

Secara formal, misalkan terdapat himpunan data $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$ dengan $\mathbf{x}_n \in \mathbb{R}^D$, dan jumlah kluster yang diinginkan adalah K . K-Means meminimalkan fungsi objektif berikut:

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2 \quad (2.50)$$

dengan $\boldsymbol{\mu}_k$ adalah *centroid* kluster ke- k , dan r_{nk} adalah variabel indikator biner yang bernilai 1 jika titik data $\mathbf{x}_n$ termasuk kluster ke- k , dan 0 jika tidak. Artinya, J mengukur total kuadrat jarak seluruh titik data ke pusat klasternya masing-masing, dan K-Means berusaha meminimalkan nilai ini.

Proses optimasi dilakukan secara iteratif melalui dua tahap yang bergantian. Pada tahap pertama (*assignment step*), setiap titik data dialokasikan ke kluster dengan *centroid terdekat*:

$$r_{nk} = \begin{cases} 1, & k = \arg \min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2 \\ 0, & \text{yang lainnya} \end{cases} \quad (2.51)$$

Pada tahap kedua (*update step*), setelah semua titik data dialokasikan, *centroid* setiap kluster diperbarui menjadi rata-rata dari seluruh titik yang menjadi anggotanya:

$$\boldsymbol{\mu}_k = \frac{\sum_{n=1}^N r_{nk} \mathbf{x}_n}{\sum_{n=1}^N r_{nk}} \quad (2.52)$$

Persamaan (2.52) dapat dipahami secara intuitif: pembilang adalah jumlah posisi seluruh anggota kluster ke- k , sedangkan penyebut adalah jumlah anggotanya, sehingga hasilnya adalah posisi rata-rata atau titik tengah dari kluster tersebut. Dua tahap ini diulang sampai nilai J tidak berubah signifikan atau jumlah iterasi mencapai batas maksimum. Algoritma lengkapnya dapat dilihat pada Tabel 2.1.

Tabel 2.1: Algoritma K-Means *Clustering*

Algoritma K-Means <i>Clustering</i>	
INPUT: Himpunan data $\{\mathbf{x}_i\}_{i=1}^N$, banyak kluster K , batas toleransi ϵ , dan maksimal iterasi maxiter	
1. Inisialisasi secara acak himpunan <i>centroid</i> $\{\boldsymbol{\mu}_j\}_{j=1}^K$	
2. Tetapkan $\text{iter} = 0$	
3. REPEAT	
4. Tetapkan $\text{iter} = \text{iter} + 1$	
5. <i>Assignment step:</i> Cari nilai $r_{nk} = \begin{cases} 1, & k = \arg \min_j \ \mathbf{x}_n - \boldsymbol{\mu}_j\ ^2 \\ 0, & \text{yang lainnya} \end{cases}$	
6. <i>Update step:</i> Cari nilai $\boldsymbol{\mu}_k = \frac{\sum_{n=1}^N r_{nk} \mathbf{x}_n}{\sum_{n=1}^N r_{nk}}$	
7. Hitung $J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \ \mathbf{x}_n - \boldsymbol{\mu}_k\ ^2$	
8. UNTIL $J < \epsilon$ OR $\text{iter} > \text{maxiter}$	
9. Untuk setiap $n = 1, 2, \dots, N$, tetapkan nilai $u_n = \arg \max_k (r_{nk})$, dengan $k = 1, 2, \dots, K$	
10. Bentuk $\{C_1, C_2, \dots, C_K\}$, dengan C_k merupakan himpunan anggota data $\mathbf{x}_i$ di mana $u_i = k$	
11. STOP	
OUTPUT: Nilai keanggotaan $\{u_n\}_{n=1}^N$ dan himpunan kluster $\{C_k\}_{k=1}^K$	

2.8 Interpretabilitas Model *Deep Learning*

Interpretabilitas model *deep learning* merujuk pada kemampuan untuk memahami dan menjelaskan keputusan yang dihasilkan oleh model yang bersifat *black-box*

(Sundararajan, Taly, and Yan 2017). Dalam konteks pemrosesan bahasa alami, interpretabilitas menjadi sangat penting karena model perlu dapat menjelaskan mengapa suatu teks diklasifikasikan ke dalam kategori tertentu, terutama pada aplikasi yang memerlukan akuntabilitas tinggi seperti analisis sentimen dan pengelompokan topik.

Secara umum, metode interpretabilitas dapat dikategorikan menjadi dua kelompok utama (J. Li *et al.* 2016). Kelompok pertama adalah metode *intrinsic*, yaitu model yang secara inheren dapat diinterpretasi seperti pohon keputusan dan regresi linier. Model jenis ini memiliki struktur yang sederhana sehingga keputusannya dapat langsung dipahami tanpa analisis tambahan. Kelompok kedua adalah metode *post-hoc*, yaitu metode yang diterapkan setelah model dilatih untuk menganalisis perilaku model yang kompleks. Metode ini tidak mengubah arsitektur model, melainkan menganalisis hubungan antara input dan output untuk menghasilkan penjelasan yang dapat dipahami manusia.

Dalam penelitian ini, metode *post-hoc* digunakan untuk menganalisis kontribusi setiap token terhadap prediksi sentimen model, yaitu *occlusion-based token attribution* dan *Integrated Gradients*.

2.8.1 Occlusion-based Token Attribution

Occlusion-based attribution merupakan salah satu metode *post-hoc* yang paling intuitif dan banyak digunakan untuk menginterpretasi model berbasis teks (Zeiler and Fergus 2014; J. Li *et al.* 2016). Metode ini berakar dari ide sederhana bahwa apabila suatu bagian dari input dihilangkan atau disembunyikan dan prediksi model berubah secara signifikan, maka bagian tersebut dianggap penting bagi keputusan model.

Pada model berbasis *transformer* seperti BERT (Devlin *et al.* 2018), oklusi dilakukan dengan mengganti token target menggunakan token khusus [MASK], sehingga model tetap menerima input dengan panjang yang sama. Pendekatan ini dikenal sebagai *leave-one-out attribution*.

Secara formal, diberikan teks input $\mathbf{x} = (x_1, x_2, \dots, x_T)$ yang terdiri dari T token, representasi kontekstual diperoleh melalui model BERT sebagai:

$$\mathbf{h} = \text{BERT}(\mathbf{x}) \in \mathbb{R}^d \quad (2.53)$$

dengan $\mathbf{h}$ merupakan representasi [CLS] token dari lapisan terakhir BERT yang berukuran d dimensi. Representasi ini kemudian diumpankan ke lapisan klasifikasi sentimen untuk menghasilkan distribusi probabilitas:

$$f(\mathbf{x}) = \text{softmax}(W \cdot \mathbf{h} + b) \in \mathbb{R}^C \quad (2.54)$$

dengan C adalah jumlah kelas sentimen, sedangkan W dan b masing-masing adalah bobot dan bias lapisan klasifikasi. Skor atribusi token x_i terhadap kelas sentimen c didefinisikan sebagai selisih probabilitas prediksi sebelum dan sesudah oklusi:

$$\text{Attr}(x_i, c) = f_c(\mathbf{x}) - f_c(\mathbf{x}_{\setminus i}) \quad (2.55)$$

dengan $f_c(\mathbf{x})$ adalah probabilitas prediksi kelas c pada teks asli, dan $f_c(\mathbf{x}_{\setminus i})$ adalah probabilitas prediksi setelah token x_i diganti dengan [MASK]. Nilai skor atribusi dapat diinterpretasikan sebagai berikut. Apabila $\text{Attr}(x_i, c) > 0$, token x_i berkontribusi positif terhadap kelas c , artinya kehadiran token tersebut meningkatkan probabilitas prediksi. Sebaliknya, apabila $\text{Attr}(x_i, c) < 0$, token tersebut justru menurunkan probabilitas prediksi kelas c . Apabila nilainya mendekati nol, yaitu $\text{Attr}(x_i, c) \approx 0$, token tersebut tidak memberikan pengaruh signifikan terhadap prediksi.

Dibandingkan dengan metode berbasis gradien, *occlusion-based attribution* memiliki keunggulan dalam hal kemudahan interpretasi karena tidak bergantung pada komputasi turunan yang kompleks. Namun, metode ini memiliki kompleksitas waktu $O(T)$ terhadap panjang token sehingga menjadi lebih mahal secara komputasi untuk teks yang panjang (J. Li *et al.* 2016).

2.8.2 Integrated Gradients

Integrated Gradients (IG) merupakan metode atribusi berbasis gradien yang diusulkan oleh Sundararajan, Taly, and Yan (2017). Berbeda dengan *occlusion-based attribution* yang bersifat diskrit, IG menghitung kontribusi setiap dimensi input secara kontinu dengan mengintegrasikan gradien prediksi model sepanjang jalur lurus dari *baseline* menuju input asli.

Secara formal, diberikan input $\mathbf{x} \in \mathbb{R}^d$ dan *baseline* $\mathbf{x}' \in \mathbb{R}^d$ yang umumnya berupa vektor nol, atribusi *Integrated Gradients* untuk dimensi ke- i didefinisikan sebagai:

$$\text{IG}_i(\mathbf{x}) = (x_i - x'_i) \times \int_{\alpha=0}^1 \frac{\partial f(\mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}'))}{\partial x_i} d\alpha \quad (2.56)$$

dengan α adalah parameter interpolasi yang menyatakan posisi sepanjang jalur dari *baseline* ke input asli, dan $\frac{\partial f}{\partial x_i}$ adalah gradien prediksi terhadap dimensi ke- i .

Dalam praktiknya, integral tersebut diaproksimasi menggunakan metode *Riemann sum* dengan m langkah interpolasi:

$$\text{IG}_i(\mathbf{x}) \approx (x_i - x'_i) \times \sum_{k=1}^m \frac{\partial f\left(\mathbf{x}' + \frac{k}{m}(\mathbf{x} - \mathbf{x}')\right)}{\partial x_i} \times \frac{1}{m} \quad (2.57)$$

Metode IG memenuhi dua aksioma penting, yaitu *sensitivity* dan *implementation invariance* (Sundararajan, Taly, and Yan 2017). *Sensitivity* menjamin bahwa apabila suatu fitur mempengaruhi prediksi model, maka fitur tersebut akan mendapatkan atribusi yang tidak nol. *Implementation invariance* menjamin bahwa dua model yang secara fungsional ekuivalen akan menghasilkan atribusi yang sama meskipun memiliki implementasi internal yang berbeda.

2.9 Interpretabilitas Model Deep Learning

Interpretabilitas model *deep learning* merujuk pada kemampuan untuk memahami dan menjelaskan keputusan yang dihasilkan oleh model yang bersifat *black-box* (Sundararajan, Taly, and Yan 2017). Dalam konteks pemrosesan bahasa alami, interpretabilitas menjadi sangat penting karena model perlu dapat menjelaskan mengapa suatu teks diklasifikasikan ke dalam kategori tertentu, terutama pada aplikasi yang memerlukan akuntabilitas tinggi seperti analisis sentimen dan pengelompokan topik.

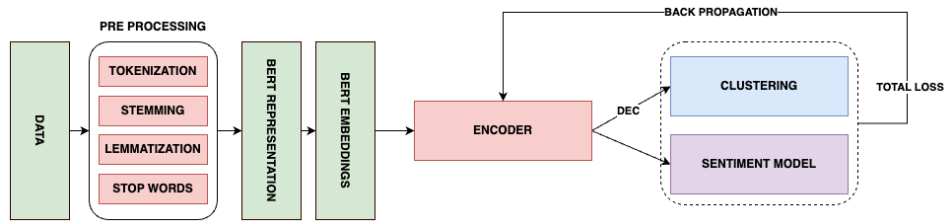
Secara umum, metode interpretabilitas dapat dikategorikan menjadi dua kelompok utama (J. Li *et al.* 2016). Kelompok pertama adalah metode *intrinsic*, yaitu model yang secara inheren dapat diinterpretasi seperti pohon keputusan dan regresi linier. Model jenis ini memiliki struktur yang sederhana sehingga keputusannya dapat langsung dipahami tanpa analisis tambahan. Kelompok kedua adalah metode *post-hoc*, yaitu metode yang diterapkan setelah model dilatih untuk menganalisis perilaku model yang kompleks. Metode ini tidak mengubah arsitektur model, melainkan menganalisis hubungan antara input dan output untuk menghasilkan penjelasan yang dapat dipahami manusia.

Dalam penelitian ini, metode *post-hoc* digunakan untuk menganalisis kontribusi setiap token terhadap prediksi sentimen model, yaitu *occlusion-based token attribution* dan *Integrated Gradients*.

BAB 3

METODE MODEL JOINT SENTIMENT-TOPIC BERBASIS MULTITASK LEARNING

Bab ini menjelaskan rancangan dan implementasi metode yang diusulkan. Pembahasan dimulai dari pemanfaatan *Bidirectional Encoder Representations from Transformers* (BERT) sebagai *feature extractor* untuk menghasilkan representasi teks, kemudian dilanjutkan dengan arsitektur model *joint sentiment-topic* berbasis *multi-task learning* yang dibangun di atasnya. Bab ini juga mencakup peran *autoencoder* dalam kompresi representasi, proses analisis sentimen menggunakan model tersebut, pengelompokan topik dengan *Deep Embedded Clustering* (DEC), interpretasi model melalui analisis atribusi token, dan terakhir evaluasi kualitas topik yang dihasilkan.



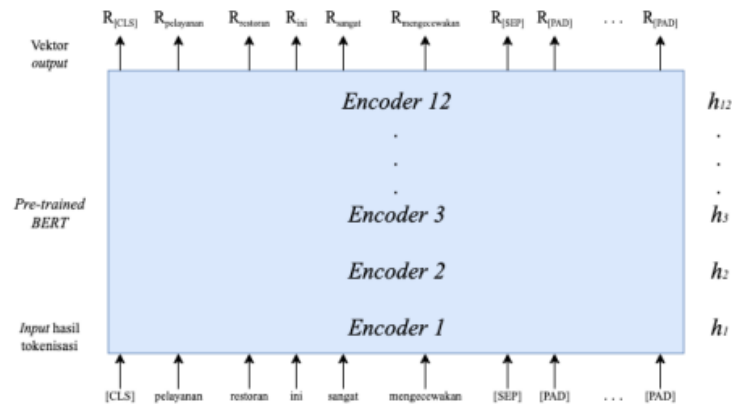
Gambar 3.1: Alur metode analisis sentimen dan pendeteksian topik

3.1 *Bidirectional Encoder Representations from Transformers* (BERT) Sebagai Model Representasi Teks

Terdapat dua pendekatan utama dalam pemanfaatan model BERT, yaitu pendekatan *feature-based* dan pendekatan *fine-tuning*. Pada pendekatan *feature-based*, BERT berfungsi sebagai *feature extractor* tanpa melakukan perubahan pada parameter internalnya. Representasi numerik berupa vektor yang dihasilkan BERT kemudian dimanfaatkan sebagai masukan bagi model lain tanpa perlu melatih ulang arsitektur BERT. Sebaliknya, pada pendekatan *fine-tuning*, BERT disesuaikan dengan menambahkan lapisan keluaran yang spesifik terhadap tugas yang diinginkan dan dilakukan pelatihan ulang secara *end-to-end* sehingga seluruh bobotnya dapat diperbarui.

Pada penelitian ini, BERT digunakan melalui pendekatan *feature-based* dengan bobot yang dibekukan sepenuhnya (*frozen*). Representasi teks yang dihasilkan

BERT kemudian dimanfaatkan sebagai masukan pada tahap berikutnya, yaitu *deep autoencoder* dan pengelompokan topik menggunakan *Deep Embedded Clustering* (DEC). Contoh representasi teks yang dihasilkan oleh BERT ditunjukkan pada Gambar 3.2.



Gambar 3.2: Representasi teks menggunakan BERT ((Devlin *et al.* 2018); telah diolah kembali).

Sebagai ilustrasi, misalkan kalimat masukan yang digunakan adalah “*pelayanan restoran ini sangat mengecewakan*”. Proses pembentukan representasi teks oleh BERT dilakukan melalui beberapa tahapan berikut.

1. **Tokenisasi.** Kalimat dipecah menjadi potongan token menggunakan *Word-Piece tokenizer* serta ditambahkan token khusus [CLS] di awal dan [SEP] di akhir, sehingga diperoleh urutan:

[[CLS], pelayanan, restoran, ini, sangat, mengecewakan,
[SEP]].

2. **Padding.** Agar panjang seluruh urutan seragam, ditambahkan token [PAD] hingga mencapai panjang maksimum yang ditetapkan, misalnya 10 token:

[[CLS], pelayanan, restoran, ini, sangat, mengecewakan,
[SEP], [PAD], [PAD], [PAD]].

3. **Attention Mask.** Dibuat *attention mask* untuk membedakan token bermakna dengan token *padding*. Token selain [PAD] diberi nilai 1 dan [PAD] diberi nilai 0, sehingga diperoleh:

$$attention\ mask = [1, 1, 1, 1, 1, 1, 1, 0, 0, 0].$$

4. **Embedding.** Urutan token diubah menjadi representasi numerik melalui lapisan *embedding*, yang merupakan hasil penjumlahan dari *token embedding*, *segment embedding*, dan *position embedding*. Representasi ini menjadi masukan awal bagi BERT untuk mempelajari konteks antar-token.
5. **Encoding.** Representasi *input* diproses melalui beberapa lapisan *encoder*, di mana setiap lapisan terdiri atas mekanisme *multi-head self-attention*, *feed-forward network*, koneksi residu, dan normalisasi. Proses ini berlangsung secara berurutan hingga lapisan *encoder* terakhir menghasilkan representasi kontekstual akhir dari seluruh token dalam kalimat.

Dalam penelitian ini, digunakan model BERT *pre-trained* yang terdiri atas 12 lapisan *encoder* dengan ukuran vektor tersembunyi (*hidden size*) sebesar 768. Misalkan t menyatakan panjang maksimum token dan n menunjukkan jumlah teks dalam *dataset*, maka model menghasilkan matriks berukuran $n \times t \times 768$ yang merepresentasikan keluaran akhir BERT untuk keseluruhan data. Pada penelitian ini, panjang maksimum token yang digunakan adalah $t = 128$.

Representasi setiap teks tidak dapat berbentuk matriks ketika digunakan sebagai masukan pada metode *clustering*, melainkan harus berupa vektor. Oleh karena itu, digunakan strategi ekstraksi token [CLS] untuk mengubah matriks keluaran berukuran $t \times 768$ menjadi vektor berdimensi 768. Token [CLS] diletakkan pada awal setiap urutan *input* dan secara eksplisit dilatih untuk merepresentasikan konteks global dari keseluruhan kalimat. Misalkan $\mathbf{H} = (h_{ij}) \in \mathbb{R}^{t \times 768}$ merupakan matriks keluaran BERT untuk suatu teks, dengan vektor baris $\mathbf{h}_i = (h_{i1}, h_{i2}, \dots, h_{i,768})$ menyatakan representasi token ke- i . Maka, vektor representasi dokumen $\mathbf{f}$ diperoleh dari baris pertama matriks $\mathbf{H}$, yaitu:

$$\mathbf{f} = \mathbf{h}_1 \in \mathbb{R}^{768} \quad (3.1)$$

Vektor $\mathbf{f}$ inilah yang selanjutnya digunakan sebagai masukan bagi *autoencoder* pada tahap berikutnya.

3.2 Model Joint Sentiment-Topic Berbasis Multitask Learning

Penelitian ini mengusulkan model *Joint Sentiment-Topic* berbasis *Multitask Learning* untuk melakukan analisis sentimen dan pendeteksian topik secara bersamaan. Model ini dibangun di atas representasi teks hasil BERT yang kemudian diproses melalui *deep autoencoder* guna memperoleh representasi laten berdimensi rendah. Representasi laten tersebut digunakan bersama oleh dua *head* pembelajaran, yaitu

sentiment head untuk klasifikasi sentimen dan *clustering head* untuk pendeteksian topik melalui DEC. Pendekatan ini mengikuti skema *hard parameter sharing*, di mana lapisan *encoder* pada *autoencoder* berfungsi sebagai jaringan bersama yang mengalirkan gradien dari kedua tugas secara simultan.

Secara keseluruhan, alur model terdiri atas tiga tahap utama. Pertama, representasi BERT berdimensi 768 untuk setiap teks diperoleh dari tahap ekstraksi fitur pada Bagian 2.5. Kedua, representasi tersebut dikompresi oleh *encoder* menjadi vektor laten $\mathbf{z}$ berdimensi lebih rendah. Ketiga, vektor laten $\mathbf{z}$ digunakan secara bersamaan oleh *sentiment head* dan *clustering head* dalam satu kerangka optimasi bersama. Total *loss* yang dioptimalkan selama pelatihan *multitask* didefinisikan sebagai:

$$\mathcal{L}_{\text{total}} = \gamma \cdot \mathcal{L}_{\text{KLD}} + \eta \cdot \mathcal{L}_{\text{CE}} \quad (3.2)$$

dengan γ dan η merupakan koefisien bobot yang mengontrol kontribusi relatif antara proses *clustering* dan klasifikasi sentimen terhadap total *loss*. Melalui optimasi bersama ini, representasi laten yang terbentuk diharapkan mampu mempertahankan struktur semantik yang relevan sekaligus untuk tugas klasifikasi maupun pengelompokan topik.

3.3 Autoencoder

Deep autoencoder umumnya sulit dilatih secara langsung karena berpotensi mengalami *overfitting* serta kesulitan menemukan representasi yang stabil. Oleh karena itu, sebelum masuk ke tahap pelatihan *multitask*, *autoencoder* terlebih dahulu dilatih secara *unsupervised* melalui proses *pretraining* dengan tujuan memberikan inisialisasi bobot awal yang baik.

Pada tahap *pretraining*, jaringan dilatih untuk merekonstruksi kembali data *input* dari representasi laten yang dihasilkan oleh *encoder*. Fungsi *loss* yang digunakan adalah *Mean Squared Error* (MSE) antara vektor *input* asli dan hasil rekonstruksinya:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{f}_i - \hat{\mathbf{f}}_i\|^2 \quad (3.3)$$

dengan N adalah jumlah total sampel, $\mathbf{f}_i$ adalah vektor representasi BERT ke- i , dan $\hat{\mathbf{f}}_i$ adalah hasil rekonstruksinya. Proses *pretraining* ini memungkinkan *autoencoder* mempelajari struktur laten yang stabil dari representasi BERT sebelum digunakan pada tahap pelatihan *multitask*.

Fungsi aktivasi yang digunakan pada sebagian besar lapisan *encoder* dan *de-*

coder adalah *Rectified Linear Unit* (ReLU), karena kemampuannya mempercepat konvergensi dan mengurangi permasalahan *vanishing gradient* pada jaringan dalam. Pada lapisan akhir *decoder*, tidak digunakan fungsi aktivasi agar hasil keluaran rekonstruksi dapat berada pada rentang nilai kontinu yang sama dengan data masukan.

Setelah *pretraining* selesai, bobot *encoder* digunakan sebagai inisialisasi untuk tahap pelatihan *multitask*. Pada tahap ini, hanya komponen *encoder* yang dipertahankan dan dioptimalkan bersama kedua *head*, sedangkan *decoder* tidak lagi digunakan. Representasi laten $\mathbf{z} = f_{\theta}(\mathbf{f})$ yang dihasilkan *encoder* menjadi masukan bersama bagi *sentiment head* untuk klasifikasi sentimen dan *clustering head* untuk pendeteksian topik menggunakan DEC.

3.4 Analisis Sentimen Menggunakan *Sentiment Head*

Vektor laten $\mathbf{z} \in \mathbb{R}^k$ yang dihasilkan oleh *encoder* diumpangkan ke lapisan *fully connected* yang berfungsi memetakan representasi tersebut ke label sentimen. Arsitektur *sentiment head* terdiri atas dua lapisan tersembunyi dengan fungsi aktivasi *Gaussian Error Linear Unit* (GELU) dan *dropout* sebagai regularisasi, diikuti oleh lapisan keluaran berdimensi dua yang merepresentasikan kelas positif dan negatif. Proses klasifikasi ini dirumuskan sebagai:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_c \mathbf{z} + \mathbf{b}_c) \quad (3.4)$$

dengan $\mathbf{W}_c$ dan $\mathbf{b}_c$ masing-masing adalah parameter bobot dan bias pada lapisan keluaran. Fungsi *softmax* mengubah nilai logit menjadi distribusi probabilitas antar kelas, di mana setiap elemen $\hat{y}_i \in [0, 1]$ dan $\sum_i \hat{y}_i = 1$.

Selama pelatihan, prediksi $\hat{\mathbf{y}}$ dibandingkan dengan label sebenarnya $\mathbf{y}$ menggunakan fungsi *loss Categorical Cross-Entropy*:

$$\mathcal{L}_{\text{CE}} = - \sum_{i=1}^n \sum_{j=1}^2 y_{ij} \log \hat{y}_{ij} \quad (3.5)$$

dengan n adalah jumlah sampel, y_{ij} adalah label aktual dalam bentuk *one-hot* bernilai 1 jika sampel ke- i termasuk kelas ke- j dan 0 jika tidak, serta $\hat{y}_{ij}$ adalah probabilitas prediksi model. Nilai $\mathcal{L}_{\text{CE}}$ ini menjadi salah satu komponen dari total *loss* pada persamaan (3.2) yang dioptimalkan secara bersama dengan komponen *clustering*.

3.5 Deep Embedded Clustering (DEC)

Tahap *Deep Embedded Clustering* (DEC) bertujuan memperbarui representasi laten $\mathbf{z}$ dan menyesuaikan posisi pusat kluster (*centroid*) agar sesuai dengan struktur distribusi data pada ruang laten (Xie, Girshick, and Farhadi 2016). Berbeda dengan pendekatan *hard assignment* yang menetapkan setiap sampel ke tepat satu kluster, DEC menggunakan distribusi probabilistik sehingga setiap sampel memiliki derajat keanggotaan terhadap seluruh kluster yang ada.

Proses DEC dimulai dengan inisialisasi *centroid* menggunakan algoritma K-Means pada representasi laten awal hasil *pretraining autoencoder*. Selanjutnya, distribusi *soft assignment* q_{ij} untuk setiap data $\mathbf{z}_i$ terhadap *centroid* μ_j dihitung menggunakan fungsi distribusi Student's t :

$$q_{ij} = \frac{\left(1 + \|\mathbf{z}_i - \mu_j\|^2\right)^{-1}}{\sum_{j'=1}^K \left(1 + \|\mathbf{z}_i - \mu_{j'}\|^2\right)^{-1}} \quad (3.6)$$

dengan q_{ij} merepresentasikan probabilitas bahwa sampel i termasuk ke dalam kluster j , dan K adalah jumlah kluster yang telah ditentukan.

Selanjutnya, DEC mendefinisikan distribusi target p_{ij} yang bersifat lebih tajam (*sharpened*) untuk meningkatkan keyakinan prediksi kluster. Misalkan $f_j = \sum_i q_{ij}$ adalah frekuensi *soft* dari kluster j , maka distribusi target diformulasikan sebagai:

$$p_{ij} = \frac{q_{ij}^2 / f_j}{\sum_{j'=1}^K q_{ij'}^2 / f_{j'}} \quad (3.7)$$

Pembagian dengan f_j berfungsi menormalisasi frekuensi kluster agar kluster yang besar tidak mendominasi, sedangkan operasi kuadrat pada q_{ij} mempertajam distribusi sehingga sampel yang ragu-ragu didorong untuk lebih memilih satu kluster. Optimasi dilakukan dengan meminimalkan *Kullback-Leibler Divergence* antara distribusi target P dan distribusi *soft assignment* Q :

$$\mathcal{L}_{\text{KLD}} = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}} \quad (3.8)$$

Nilai $\mathcal{L}_{\text{KLD}}$ ini digunakan sebagai sinyal kesalahan untuk memperbarui parameter *encoder* dan posisi *centroid* secara bersamaan menggunakan algoritma optimasi Adam. Proses pembaruan dilakukan secara iteratif; pada setiap *update interval*, label kluster dari iterasi sebelumnya dibandingkan dengan yang baru, dan pelati-

han dihentikan apabila perubahan label antar iterasi (Δ_{label}) lebih kecil dari ambang batas toleransi τ yang ditetapkan. Prosedur pelatihan DEC secara lengkap dirangkum pada Tabel 3.1.

Tabel 3.1: Algoritma *Deep Embedded Clustering* (DEC)

Algoritma <i>Deep Embedded Clustering</i> (DEC)	
INPUT	Himpunan vektor data $\{\mathbf{f}_i\}_{i=1}^N \in \mathbb{R}^{768}$; jumlah kluster K ; batch size b ; ambang toleransi konvergensi τ ; parameter bobot loss γ, η ; jumlah iterasi maksimum $T_{\max}$
OUTPUT	Representasi laten $\{\mathbf{z}_i\}$; label kluster $\{y_{\text{cluster}}\}$; label sentimen $\{y_{\text{sentiment}}\}$
1.	Latih <i>autoencoder</i> dengan meminimalkan fungsi rekonstruksi MSE: $\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \ \mathbf{f}_i - \hat{\mathbf{f}}_i\ ^2$
2.	Ambil bagian <i>encoder</i> untuk memperoleh representasi laten $\mathbf{z}_i = f_{\theta}(\mathbf{f}_i)$
3.	Inisialisasi <i>centroid</i> awal $\{\boldsymbol{\mu}_j\}_{j=1}^K$ menggunakan K-Means pada $\{\mathbf{z}_i\}$
4.	FOR iter = 1 hingga $T_{\max}$ 4.1 Simpan label kluster sebelumnya y_{prev} 4.2 Proyeksikan data ke ruang laten: $\mathbf{z}_i = f_{\theta}(\mathbf{f}_i)$ 4.3 Hitung <i>soft assignment</i> q_{ij} menggunakan distribusi Student's t 4.4 Hitung distribusi target p_{ij} berdasarkan q_{ij} 4.5 Hitung keluaran sentimen $\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_c \mathbf{z} + \mathbf{b}_c)$ 4.6 Perbarui parameter model dengan meminimalkan <i>joint loss</i>: $\mathcal{L}_{\text{total}} = \gamma \cdot \mathcal{L}_{\text{KLD}}(P, Q) + \eta \cdot \mathcal{L}_{\text{CE}}(\mathbf{y}, \hat{\mathbf{y}})$ 4.7 Jika $\Delta_{\text{label}} < \tau$, hentikan pelatihan
5.	END FOR
6.	Simpan bobot terbaik model berdasarkan metrik validasi
OUTPUT:	Representasi laten $\{\mathbf{z}_i\}$, label kluster $\{y_{\text{cluster}}\}$, dan label sentimen $\{y_{\text{sentiment}}\}$

3.6 Interpretasi Model dengan Analisis Atribusi Token

Untuk memperoleh pemahaman yang lebih mendalam mengenai keputusan model, penelitian ini menerapkan dua metode interpretasi *post-hoc*, yaitu *occlusion-based token attribution* pada level token dan *Integrated Gradients* pada level dimensi *embedding*. Kedua metode ini diterapkan setelah proses pelatihan selesai menggunakan bobot model terbaik.

3.6.1 Implementasi *Occlusion-based Token Attribution*

Proses *occlusion-based token attribution* dilakukan dengan memanfaatkan model BERT sebagai *feature extractor*. Alur komputasi atribusi token adalah sebagai

berikut. Teks *input* $\mathbf{x}$ diumpankan ke BERT untuk mendapatkan representasi token $[\text{CLS}] \mathbf{h} \in \mathbb{R}^d$, yang kemudian diumpankan ke *sentiment head* untuk mendapatkan probabilitas prediksi *baseline* $f_c(\mathbf{x})$ terhadap kelas sentimen c . Selanjutnya, untuk setiap token x_i dalam teks (kecuali token spesial $[\text{CLS}]$, $[\text{SEP}]$, dan $[\text{PAD}]$), token tersebut diganti dengan $[\text{MASK}]$ untuk menghasilkan teks yang dimodifikasi $\mathbf{x}_{\setminus i}$. Teks yang dimodifikasi diumpankan kembali ke BERT dan *sentiment head* untuk mendapatkan probabilitas prediksi $f_c(\mathbf{x}_{\setminus i})$. Skor atribusi token x_i terhadap kelas sentimen c kemudian dihitung sebagai selisih probabilitas sebelum dan sesudah oklusi:

$$\text{Attr}(x_i, c) = f_c(\mathbf{x}) - f_c(\mathbf{x}_{\setminus i}) \quad (3.9)$$

Nilai $\text{Attr}(x_i, c) > 0$ berarti token x_i berkontribusi positif terhadap kelas c , nilai negatif berarti token tersebut menurunkan probabilitas prediksi kelas c , dan nilai mendekati nol berarti token tidak memberikan pengaruh signifikan.

3.6.2 Agregasi Atribusi Token per Klaster Topik

Untuk memperoleh pemahaman pola linguistik pada level klaster, skor atribusi token diagregasi per klaster. Diberikan himpunan sampel pada klaster k sebagai $C_k = \{i : z_i = k\}$ di mana z_i adalah label klaster dari sampel ke- i , skor atribusi rata-rata token w pada klaster k didefinisikan sebagai:

$$\overline{\text{Attr}}(w, k, c) = \frac{1}{|C_k|} \sum_{i \in C_k} \text{Attr}(x_i^{(w)}, c) \quad (3.10)$$

dengan $x_i^{(w)}$ merujuk pada posisi token w dalam sampel ke- i dan $|C_k|$ adalah jumlah sampel dalam klaster k . Untuk memastikan reliabilitas estimasi, hanya token yang muncul pada minimal dua sampel dalam suatu klaster yang diikutsertakan dalam perhitungan. Himpunan token yang memenuhi syarat ini didefinisikan sebagai:

$$\mathcal{V}_k = \left\{ w \left| \sum_{i \in C_k} \mathbf{1}[w \in \mathbf{x}_i] \geq 2 \right. \right\} \quad (3.11)$$

Token-token dalam $\mathcal{V}_k$ kemudian diurutkan berdasarkan nilai absolut skor atribusi rata-rata untuk memperoleh daftar K token paling berpengaruh pada klaster k :

$$\text{TopTokens}(k, c) = \arg \text{top-}K \left| \overline{\text{Attr}}(w, k, c) \right|_{w \in \mathcal{V}_k} \quad (3.12)$$

Selain itu, pada implementasinya, kosakata token yang dievaluasi dibatasi hanya pada kata-kata yang termasuk dalam top- N TF-IDF dari masing-masing klaster. Pembatasan ini memastikan bahwa skor atribusi yang dikumpulkan hanya berasal dari kata-kata yang memang khas dan representatif untuk klaster tersebut, sehingga hasil interpretasi lebih bermakna dan tidak terkontaminasi oleh token yang tidak relevan.

3.6.3 Implementasi *Integrated Gradients*

Pada penelitian ini, *Integrated Gradients* diterapkan pada level dimensi *embedding* untuk mengidentifikasi dimensi-dimensi yang paling berpengaruh terhadap prediksi sentimen. *Baseline* yang digunakan adalah vektor nol $\mathbf{x}' = \mathbf{0} \in \mathbb{R}^d$, dan jumlah langkah interpolasi yang digunakan adalah $m = 50$ langkah sebagaimana direkomendasikan oleh Sundararajan, Taly, and Yan (2017). Atribusi IG untuk setiap dimensi *embedding* diaproksimasi menggunakan *Riemann sum*:

$$\text{IG}_i(\mathbf{x}) \approx (x_i - x'_i) \times \sum_{k=1}^m \frac{\partial f(\mathbf{x}' + \frac{k}{m}(\mathbf{x} - \mathbf{x}'))}{\partial x_i} \times \frac{1}{m} \quad (3.13)$$

dengan x_i adalah nilai dimensi ke- i dari *embedding* masukan, x'_i adalah nilai *baseline* pada dimensi yang sama, dan $\frac{\partial f}{\partial x_i}$ adalah gradien prediksi model terhadap dimensi ke- i . Atribusi IG kemudian diagregasi per klaster untuk mengidentifikasi dimensi mana yang paling berperan dalam membedakan satu klaster dengan lainnya:

$$\overline{\text{IG}}_i(k) = \frac{1}{|C_k|} \sum_{j \in C_k} |\text{IG}_i(\mathbf{x}_j)| \quad (3.14)$$

dengan $\overline{\text{IG}}_i(k)$ adalah rata-rata nilai absolut atribusi IG dimensi ke- i pada klaster k , dan $\mathbf{x}_j$ adalah *embedding* dari sampel ke- j dalam klaster k . Hasil agregasi ini divisualisasikan dalam bentuk *heatmap* untuk memudahkan analisis komparatif antar klaster.

3.7 Evaluasi Topik

Evaluasi kualitas topik yang dihasilkan oleh model dilakukan menggunakan tiga metrik kuantitatif, yaitu *topic coherence*, *topic diversity*, dan *topic coverage*. Ketiga metrik ini dihitung secara langsung dari teks asli dan hasil pengelompokan klaster tanpa bergantung pada model eksternal, sehingga bersifat *self-contained* dan dapat dijalankan sepenuhnya dalam kerangka pelatihan model.

3.7.1 Topic Coherence Berbasis NPMI

Topic coherence digunakan untuk mengukur sejauh mana kata-kata teratas yang merepresentasikan suatu topik memiliki keterkaitan semantik yang nyata dalam korpus. Metrik yang digunakan adalah *Normalized Pointwise Mutual Information* (NPMI), yang mengkuantifikasi tingkat ko-okurens antar pasangan kata pada teks-teks dalam kluster yang sama.

Misalkan $\mathcal{D}_k$ adalah himpunan teks dalam kluster k dan $\{w_1, w_2, \dots, w_N\}$ adalah N kata teratas dari kluster tersebut berdasarkan frekuensi kemunculan setelah pemfilteran *stop words*. Untuk setiap pasangan kata (w_i, w_j) dengan $i < j$, probabilitas kemunculan masing-masing kata dan ko-okurensnya didefinisikan sebagai:

$$p(w_i) = \frac{|\{d \in \mathcal{D}_k : w_i \in d\}|}{|\mathcal{D}_k|}, \quad (3.15)$$

$$p(w_j) = \frac{|\{d \in \mathcal{D}_k : w_j \in d\}|}{|\mathcal{D}_k|}, \quad (3.16)$$

$$p(w_i, w_j) = \frac{|\{d \in \mathcal{D}_k : w_i \in d \text{ dan } w_j \in d\}|}{|\mathcal{D}_k|}. \quad (3.17)$$

Nilai *Pointwise Mutual Information* (PMI) antara dua kata dihitung sebagai:

$$\text{PMI}(w_i, w_j) = \log \frac{p(w_i, w_j) + \epsilon}{p(w_i) \cdot p(w_j) + \epsilon} \quad (3.18)$$

dengan $\epsilon = 10^{-10}$ adalah konstanta kecil untuk mencegah pembagian dengan nol. Selanjutnya, PMI dinormalisasi menggunakan probabilitas ko-okurens sebagai pembagi:

$$\text{NPMI}(w_i, w_j) = \frac{\text{PMI}(w_i, w_j)}{-\log(p(w_i, w_j) + \epsilon)} \quad (3.19)$$

Nilai NPMI berada pada rentang $[-1, 1]$, di mana nilai mendekati 1 menandakan ko-okurens yang sangat tinggi, nilai 0 menandakan independensi, dan nilai negatif menandakan kata-kata yang cenderung tidak muncul bersama. Skor *topic coherence* untuk kluster k diperoleh dengan merata-ratakan seluruh nilai NPMI atas semua pasangan kata teratas:

$$\text{TC}_k = \frac{1}{\binom{N}{2}} \sum_{j=2}^N \sum_{i=1}^{j-1} \text{NPMI}(w_i, w_j) \quad (3.20)$$

Skor koherensi keseluruhan model diperoleh dengan merata-ratakan nilai TC_k

di seluruh klaster:

$$\overline{\text{TC}} = \frac{1}{K} \sum_{k=1}^K \text{TC}_k \quad (3.21)$$

Metrik ini hanya dihitung apabila suatu klaster memiliki setidaknya dua kata teratas yang unik; klaster dengan jumlah kata kurang dari dua diberikan nilai koherensi 0.

3.7.2 *Topic Diversity*

Topic diversity mengukur sejauh mana setiap klaster memiliki kosakata yang berbeda satu sama lain sehingga tidak terjadi tumpang tindih antar topik. Misalkan $\mathcal{V}_k$ adalah himpunan N kata teratas dari klaster k . Himpunan kata unik di seluruh klaster didefinisikan sebagai $\mathcal{U} = \bigcup_{k=1}^K \mathcal{V}_k$, sedangkan total kata dengan duplikat adalah $\sum_{k=1}^K |\mathcal{V}_k|$. *Topic diversity* didefinisikan sebagai:

$$\text{TD} = \frac{|\mathcal{U}|}{\sum_{k=1}^K |\mathcal{V}_k|} \quad (3.22)$$

Nilai $\text{TD} = 1$ menunjukkan bahwa seluruh kata teratas dari semua klaster bersifat unik, sedangkan nilai yang rendah mengindikasikan adanya tumpang tindih kosakata antar topik. Nilai TD yang tinggi bersama-sama dengan $\overline{\text{TC}}$ yang tinggi mencerminkan model yang mampu menghasilkan topik-topik yang tidak hanya koheren secara internal, tetapi juga saling berbeda secara leksikal.

3.7.3 *Topic Coverage dan Keseimbangan Klaster*

Topic coverage mengevaluasi sejauh mana distribusi data antar klaster bersifat seimbang. Ketidakseimbangan yang ekstrem, di mana satu klaster mendominasi sementara klaster lain hampir kosong, mengindikasikan bahwa model gagal mengeksplorasi keragaman semantik data secara menyeluruh.

Misalkan $c_k = |\{i : y_i = k\}|$ adalah jumlah dokumen pada klaster k dan $p_k = c_k/N$ adalah proporsinya terhadap total N dokumen. Keseimbangan klaster diukur menggunakan entropi Shannon yang dinormalisasi:

$$\text{CB} = \frac{H}{\log K}, \quad H = - \sum_{k=1}^K p_k \log p_k \quad (3.23)$$

dengan $\log K$ merupakan nilai entropi maksimum yang dicapai ketika seluruh klaster memiliki ukuran yang sama. Nilai $\text{CB} \in [0, 1]$; nilai mendekati 1

menandakan distribusi yang seimbang dan nilai mendekati 0 menandakan distribusi yang sangat tidak merata. Selain itu, dilaporkan pula proporsi klaster terkecil $p_{\min} = \min_k p_k$ dan terbesar $p_{\max} = \max_k p_k$, di mana selisih antara keduanya yang besar mengindikasikan distribusi yang tidak seimbang.

BAB 4
HASIL DAN SIMULASI DAN PEMBAHASAN

BAB 5

KESIMPULAN

@todo

Tambahkan kesimpulan dan saran terkait dengan pekerjaan yang dilakukan.

5.1 Kesimpulan

5.2 Saran

DAFTAR PUSTAKA

- [1] Charu C. Aggarwal. *Neural Networks and Deep Learning: A Textbook*. Cham: Springer, 2018. ISBN: 978-3-319-94462-3. DOI: 10.1007/978-3-319-94463-0.
- [2] James Allan. *Topic Detection and Tracking: Event-Based Information Organization*. Vol. 12. The Information Retrieval Series. New York: Springer, 2002. ISBN: 978-0-7923-7664-4.
- [3] Dima Angelov. “Top2Vec: Distributed Representations of Topics”. *arXiv preprint arXiv:2008.09470* (2020).
- [4] Jonathan Baxter. “A Model of Inductive Bias Learning”. *Journal of Artificial Intelligence Research* 12 (2000), pp. 149–198. DOI: 10.1613/jair.731.
- [5] James C. Bezdek. *Pattern Recognition with Fuzzy Objective Function Algorithms*. New York: Springer Science & Business Media, 1984. ISBN: 978-1-4757-0450-1. DOI: 10.1007/978-1-4757-0450-1.
- [6] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. New York: Springer, 2006. ISBN: 978-0-387-31073-2.
- [7] David M. Blei. “Probabilistic topic models”. *Communications of the ACM* 55.4 (2012), pp. 77–84.
- [8] David M. Blei, Andrew Y. Ng, and Michael I. Jordan. “Latent Dirichlet allocation”. *Journal of Machine Learning Research* 3 (2003), pp. 993–1022.
- [9] Jordan Boyd-Graber, Yuen Hu, and David Mimno. “Applications of topic models”. *Foundations and Trends in Information Retrieval* 11.2–3 (2017), pp. 143–296.
- [10] Ricardo J. G. B. Campello, Davoud Moulavi, and Jörg Sander. “Density-Based Clustering Based on Hierarchical Density Estimates”. *Advances in Knowledge Discovery and Data Mining* (2013), pp. 160–172.
- [11] Dallas Card, Chenhao Tan, and Noah A. Smith. “A Neural Framework for Generalized Topic Models”. *arXiv preprint arXiv:1705.09296* (2017).
- [12] Rich Caruana. “Multitask Learning”. *Machine Learning* 28 (1997), pp. 41–75. DOI: 10.1023/A:1007379606734.

- [13] Huimin Chen *et al.* “Neural Sentiment Classification with User and Product Attention”. *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2016, pp. 1650–1859.
- [14] Kyunghyun Cho *et al.* “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation”. *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2014, pp. 1724–1734.
- [15] Junyoung Chung *et al.* “Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling”. *arXiv preprint arXiv:1412.3555* (2014).
- [16] Ronan Collobert *et al.* “Natural Language Processing (Almost) from Scratch”. *Journal of Machine Learning Research* 12 (2011), pp. 2493–2537.
- [17] Jacob Devlin *et al.* “BERT: Pre-training of deep bidirectional transformers for language understanding”. *arXiv preprint arXiv:1810.04805* (2018).
- [18] Adji B. Dieng, Francisco J. R. Ruiz, and David M. Blei. “Topic Modeling in Embedding Spaces”. *Transactions of the Association for Computational Linguistics* 8 (2020), pp. 439–453.
- [19] Long Duong *et al.* “Low Resource Dependency Parsing: Cross-lingual Parameter Sharing in a Neural Network Parser” (2015), pp. 845–850.
- [20] Martin Ester *et al.* “A Density-based Algorithm for Discovering Clusters in Large Spatial Databases with Noise”. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*. 1996, pp. 226–231.
- [21] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. Cambridge, MA: MIT Press, 2016. ISBN: 978-0-262-03561-3. URL: <http://www.deeplearningbook.org>.
- [22] Alex Graves. “Generating Sequences with Recurrent Neural Networks”. *arXiv preprint arXiv:1308.0850* (2013).
- [23] Maarten Grootendorst. “BERTopic: Neural topic modeling with a class-based TF-IDF procedure”. *arXiv preprint arXiv:2203.05794* (2022).
- [24] Lin Gui *et al.* “Multi-Task Mutual Learning for Joint Sentiment Classification and Topic Detection”. *IEEE Transactions on Knowledge and Data Engineering* 34.4 (2022), pp. 1915–1927.

- [25] Xifeng Guo *et al.* “Improved Deep Embedded Clustering with Local Structure Preservation”. *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*. 2017, pp. 1753–1759. DOI: 10.24963/ijcai.2017/243. URL: <https://www.ijcai.org/proceedings/2017/0243.pdf>.
- [26] Dan Hendrycks and Kevin Gimpel. “Gaussian Error Linear Units (GELUs)”. *arXiv preprint arXiv:1606.08415* (2016). URL: <https://arxiv.org/abs/1606.08415>.
- [27] Geoffrey E. Hinton and Ruslan R. Salakhutdinov. “Reducing the Dimensionality of Data with Neural Networks”. *Science* 313.5786 (2006), pp. 504–507. DOI: 10.1126/science.1127647.
- [28] Yohan Jo and Alice H. Oh. “Aspect and Sentiment Unification Model for Online Review Analysis”. *Proceedings of the Fourth ACM International Conference on Web Search and Data Mining*. 2011, pp. 815–824.
- [29] Yoon Kim. “Convolutional Neural Networks for Sentence Classification”. *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2014, pp. 1746–1751.
- [30] Gary King and Langche Zeng. “Logistic Regression in Rare Events Data”. *Political Analysis* 9.2 (2001), pp. 137–163.
- [31] Diederik P. Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. Published as a conference paper at ICLR 2015. 2015. URL: <https://arxiv.org/abs/1412.6980>.
- [32] Diederik P. Kingma and Max Welling. “Auto-Encoding Variational Bayes”. *arXiv preprint arXiv:1312.6114* (2013).
- [33] Daniel D. Lee and H. Sebastian Seung. “Learning the Parts of Objects by Non-Negative Matrix Factorization”. *Nature* 401.6755 (1999), pp. 788–791. DOI: 10.1038/44565.
- [34] Jiwei Li *et al.* “Visualizing and Understanding Neural Models in NLP”. *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. Association for Computational Linguistics, 2016, pp. 681–691.
- [35] Qianqian Liang *et al.* “JST-RR model: Joint modeling of ratings and reviews in sentiment-topic prediction”. *arXiv preprint arXiv:2102.11048* (2021).

- [36] Chong Lin and Yulan He. “Joint sentiment/topic model for sentiment analysis”. *Proceedings of the 18th ACM Conference on Information and Knowledge Management (CIKM)*. 2009, pp. 375–384.
- [37] Bing Liu. *Sentiment Analysis and Opinion Mining*. Synthesis Lectures on Human Language Technologies. Morgan & Claypool Publishers, 2012. ISBN: 978-1-60845-884-4.
- [38] Pengfei Liu, Xipeng Qiu, and Xuanjing Huang. “Adversarial Multi-task Learning for Text Classification”. *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*. 2017, pp. 1–10.
- [39] Pengfei Liu, Xipeng Qiu, and Xuanjing Huang. “Recurrent Neural Network for Text Classification with Multi-Task Learning”. *Proceedings of the 25th International Joint Conference on Artificial Intelligence*. 2016, pp. 2873–2879.
- [40] Andrew L. Maas *et al.* “Learning Word Vectors for Sentiment Analysis”. *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*. 2011, pp. 142–150.
- [41] Pradipta Maji and Sankar K. Pal. “Rough Set Based Generalized Fuzzy C-Means Algorithm and Quantitative Indices”. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* 37.6 (2007). Cited as Maji *et al.*, 2002 for soft/hard clustering concepts, pp. 1529–1540. DOI: 10.1109/TSMCB.2007.906578.
- [42] Julian McAuley *et al.* “Image-based Recommendations on Styles and Substitutes”. *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2015, pp. 43–52.
- [43] Jon D. McAuliffe and David M. Blei. “Supervised topic models”. *Advances in Neural Information Processing Systems*. 2008, pp. 121–128.
- [44] Qiaozhu Mei *et al.* “Topic Sentiment Mixture: Modeling Facets and Opinions in Weblogs”. *Proceedings of the 16th International Conference on World Wide Web*. 2007, pp. 171–180.
- [45] Yishu Miao, Lei Yu, and Phil Blunsom. “Neural Variational Inference for Text Processing”. *Proceedings of the International Conference on Machine Learning*. 2016, pp. 1727–1736.
- [46] Tomas Mikolov, Kai Chen *et al.* “Efficient estimation of word representations in vector space”. *arXiv preprint arXiv:1301.3781* (2013).

- [47] Tomas Mikolov, Ilya Sutskever *et al.* “Distributed Representations of Words and Phrases and Their Compositionality”. *Proceedings of Advances in Neural Information Processing Systems*. 2013, pp. 3111–3119.
- [48] Shruti Pandit and Shikha Gupta. “A Comparative Study on Distance Measuring Approaches for Clustering”. *International Journal of Research in Computer Science* 2.1 (2011), pp. 29–31. DOI: 10.7815/ijorcs.21.2011.011.
- [49] Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan. “Thumbs up? Sentiment Classification using Machine Learning Techniques”. *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2002, pp. 79–86.
- [50] Michael Röder, Andreas Both, and Alexander Hinneburg. “Exploring the Space of Topic Coherence Measures”. *Proceedings of the 8th ACM International Conference on Web Search and Data Mining*. 2015, pp. 399–408.
- [51] Sebastian Ruder. “An Overview of Multi-Task Learning in Deep Neural Networks”. *arXiv preprint arXiv:1706.05098* (2017). URL: <https://arxiv.org/abs/1706.05098>.
- [52] Gerard Salton and Christopher Buckley. “Term-weighting approaches in automatic text retrieval”. *Information Processing & Management* 24.5 (1988), pp. 513–523.
- [53] Ivan Nunes da Silva *et al.* *Artificial Neural Networks: A Practical Course*. Cham: Springer, 2017. ISBN: 978-3-319-43162-8. DOI: 10.1007/978-3-319-43162-8.
- [54] Akash Srivastava and Charles Sutton. “Autoencoding Variational Inference for Topic Models”. *Proceedings of the International Conference on Learning Representations*. 2017.
- [55] Chang Sun *et al.* “Fine-tuning BERT for joint entity and relation extraction in Chinese medical text”. *Proceedings of the 12th International Conference on Knowledge Science, Engineering and Management*. 2019, pp. 197–207.
- [56] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. “Axiomatic Attribution for Deep Networks”. *Proceedings of the 34th International Conference on Machine Learning (ICML)*. PMLR, 2017, pp. 3319–3328.
- [57] Duyu Tang, Bing Qin, and Ting Liu. “Aspect Level Sentiment Classification with Deep Memory Network”. *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2016, pp. 214–224.

- [58] Duyu Tang, Bing Qin, and Ting Liu. “Document Modeling with Gated Recurrent Neural Network for Sentiment Classification”. *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2015, pp. 1422–1432.
- [59] Ashish Vaswani *et al.* “Attention is all you need”. *Advances in Neural Information Processing Systems*. 2017, pp. 5998–6008.
- [60] Pascal Vincent *et al.* “Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion”. *Journal of Machine Learning Research* 11 (2010), pp. 3371–3408.
- [61] Sida Wang and Christopher D. Manning. “Baselines and Bigrams: Simple, Good Sentiment and Topic Classification”. *Proceedings of the 50th Annual Meeting of the Association for Computational Linguistics*. 2012, pp. 90–94.
- [62] Junyuan Xie, Ross Girshick, and Ali Farhadi. “Unsupervised deep embedding for clustering analysis”. *International Conference on Machine Learning (ICML)*. 2016.
- [63] Zichao Yang *et al.* “Hierarchical Attention Networks for Document Classification”. *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2016, pp. 1480–1489.
- [64] Hendi Yusdiansyah and Kuspriyanto Suryadi. “Centroid-Based Clustering for Text Classification”. *Journal of Physics: Conference Series* 1192.1 (2019), p. 012022. DOI: 10.1088/1742-6596/1192/1/012022.
- [65] Matthew D. Zeiler and Rob Fergus. “Visualizing and Understanding Convolutional Networks”. *European Conference on Computer Vision (ECCV)*. Springer, 2014, pp. 818–833.
- [66] Aston Zhang *et al.* *Dive into Deep Learning*. Cambridge University Press, 2023. ISBN: 978-1-009-38943-3. URL: <https://d2l.ai>.
- [67] Xiang Zhang, Junbo Zhao, and Yann LeCun. “Character-level Convolutional Networks for Text Classification”. *Advances in Neural Information Processing Systems*. 2015, pp. 649–657.
- [68] Yinhan Zhang, Hao Zhao, and Ming Li. “Neural topic models: A survey”. *arXiv preprint arXiv:2004.11906* (2020).
- [69] Yongliang Zhang *et al.* “Multi-task mutual learning for joint sentiment classification and topic detection”. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 28 (2020), pp. 2407–2417.

- [70] Yu Zhang and Qiang Yang. “A Survey on Multi-Task Learning”. *arXiv preprint arXiv:1707.08114* (2017).
- [71] Zhibin Zhou *et al.* “A comprehensive survey on clustering: From traditional methods to modern deep learning algorithms”. *arXiv preprint arXiv:2102.08012* (2021).